Software-as-a-Graph: Heterogeneous Graph Learning for Pre-Deployment Reliability and Dependability Analysis of Complex Distributed Systems

Abstract

Modern distributed software—such as publish–subscribe middleware and microservices—heavily decouples components to achieve scalability. However, this decoupling obscures how cascading failures propagate across message brokers, topics, shared libraries, and host nodes. Detecting systemically critical components before deployment is challenging: operational telemetry does not yet exist, and traditional static code analysis cannot observe distributed communication topologies. When uncontained cascading failures reach production, they trigger costly downtime, emergency restart loops, and excessive energy consumption.

To solve this, we present **Software-as-a-Graph (SaG)**, an AI-driven framework for pre-deployment reliability analysis. SaG represents distributed system architectures as typed multigraphs across five core entities: Applications, Brokers, Topics, Execution Nodes, and Shared Libraries. On this representation, SaG pairs two complementary techniques: **(1)** a relation-specific **Heterogeneous Graph Transformer (HGL)** that learns multi-layer propagation patterns to forecast cascading failure blast radii and rank critical components; and **(2)** an **Explainable Quality Attribution (RM)** baseline grounded in the ISO/IEC 25010 and ISO/IEC 25019 quality models that explains the structural root causes of vulnerability.

We evaluate SaG across 1,770 components from seven synthetic scenarios and three real-world open-source systems (Autoware.universe ROS 2, GCP Cloud Microservices, and the Train-Ticket benchmark) against independent discrete-event cascade simulators under a strict input–label independence guarantee. Our key findings show:

- **Value of Heterogeneous Modeling:** Relation-specific typing is what makes graph learning transfer: on unseen architectures the typed model reaches $\rho = 0.608$ where its homogeneous counterpart collapses to 0.086, a +0.365 gap won in all eight cross-scenario folds ($p = 0.008$). Quality-of-Service edge encoding contributes a further +0.157 out of distribution (7/8 folds, $p = 0.016$) at a small, statistically insignificant in-distribution cost.
- **An Explicit Boundary:** Against a training-free QoS-weighted structural baseline, the typed model's ranking advantage is +0.037 and *not* statistically significant ($p = 0.74$). We report this directly: graph learning here earns its cost through the typed attention, multi-task quality outputs, and relationship-level criticality it supplies alongside the ranking, not through ranking accuracy alone.
- **Actionable Explainability:** The quality attribution model successfully separates single points of failure (availability) from error propagation depth (fault tolerance), turning black-box graph predictions into concrete architectural fixes (e.g., broker replication vs. topic decoupling).
- **Real-World Generalization:** SaG transfers effectively to authentic cyber-physical and cloud-native systems, achieving high rank agreement ($\rho = 0.688$ to 0.778) and capturing up to 100% of failure-impactful services within the predicted critical set.

By enabling automated architectural analysis within CI/CD pipelines before deployment, SaG bridges the Architecture–Code Gap, fostering resilient, dependable, and energy-efficient software systems.

Keywords: Graph representation learning, Heterogeneous graph neural networks, Publish–subscribe middleware, Distributed systems dependability, Cascading failures, Static system analysis, Architectural quality models

1. Introduction

1.1. Motivation

Modern large-scale distributed systems increasingly rely on asynchronous, event-driven, and publish–subscribe (pub-sub) architectures. From autonomous vehicles (ROS 2 [1]) and high-throughput enterprise backbones (Apache Kafka [2]) to cyber-physical systems (DDS [3]) and IoT meshes (MQTT [4]), pub-sub decouples producers and consumers in space, time, and synchronization [5]. Components communicate indirectly by sending and receiving messages through intermediate topics and brokers without maintaining direct references to one another. Furthermore, modern middleware lets developers specify deployment-time Quality-of-Service (QoS) policies—such as message durability, transport reliability, and delivery deadlines—to control how data flows under stress.

While this decoupling makes distributed systems scalable and flexible, it introduces a severe **visibility barrier** for system reliability and dependability:

- **Indirect Failure Pathways:** In traditional synchronous systems (such as REST or RPC architectures), component interactions follow explicit caller–callee call graphs. In pub-sub and event-driven meshes, there are no direct static references between publishers and subscribers. Cascading failures propagate across hidden logical pathways spanning brokers, shared topics, colocated execution nodes, and shared software libraries.

- **Distinct Failure Mechanisms:** Failures in distributed systems do not propagate in a single way. They manifest as either *sequential cascades* (e.g., slow consumer message-queue backlogs propagating downstream) or *simultaneous blast radii* (e.g., a shared library crash or node failure instantly disabling multiple colocated services at the same moment). Traditional architectural diagrams and static call graphs fail to capture these multi-layer interactions.

Fixing these vulnerabilities is easiest and cheapest **prior to deployment**, during architectural design and Continuous Integration / Continuous Delivery (CI/CD). However, pre-deployment is precisely when **no runtime telemetry, distributed tracing, or operational logs exist**. As a result, software architects and reliability engineers face two fundamental questions without runtime data:

1. *Which components and communication links are systemically critical to system reliability and availability?*
2. *Why are they critical, and what specific architectural fix (such as replicating a broker, decoupling a shared topic, or sandboxing a library) will most effectively eliminate that risk?*

Addressing this challenge is also critical for **system performance and sustainability**. When cascading failures occur in production, they trigger compute-intensive restart loops, failover storms, and retransmissions. Proactive, design-time architectural hardening prevents these failures before software is deployed, saving infrastructure costs and eliminating wasted energy.

1.2. Problem Statement: The Architecture–Code Gap

We formulate pre-deployment dependability analysis as two distinct, complementary tasks spanning qualitative diagnosis and quantitative forecasting:

1. **Explainable Criticality Attribution (Diagnostic Path):** Computing an interpretable, standards-grounded structural quality profile for every component—grounded in ISO/IEC 25010 [6] and ISO/IEC 25019 [7]—to diagnose the *qualitative nature* of a component's vulnerability, for example distinguishing a single point of failure from a high-coupling maintenance bottleneck. This task answers *why* a component is fragile and which remediation applies; it is not a predictive ranking model, and we do not evaluate it as one.
2. **Failure-Impact Forecasting (Predictive Path):** Forecasting the dynamic, global cascading failure blast radius and ranking the systemically critical component set, by training a data-driven, non-linear model over learned topological representations.

The separation is architectural, not merely presentational: the two paths consume the same graph but share no parameters, and neither is fitted to the other's output. Keeping them distinct is what allows a component to be reported as, say, structurally central but operationally low-impact — a diagnosis neither path produces alone.

Existing software engineering approaches fail to bridge what we define as the **"Architecture–Code Gap"**: *a distributed system can have pristine, 100% bug-free source code within each service, yet remain fragile to catastrophic global outages due to hidden architectural single points of failure (SPOFs) or mismatched middleware QoS contracts.* Three prevailing paradigms leave this gap unaddressed:

- **Static Code Analysis (SCA):** Tools like SonarQube inspect source code complexity [8], modularity, and cohesion [9, 10] within single services. However, SCA cannot see the broader distributed network, message queues, or cross-host failure propagation.
- **Runtime Chaos Engineering:** Techniques like Chaos Monkey [11] and distributed tracing inject real faults into running staging or production environments. While effective at validating live systems, they require fully deployed infrastructure, carry operational risks, and arrive too late to guide initial architectural design.
- **Homogeneous Graph Centrality Metrics:** Standard network metrics (betweenness, PageRank, degree) [12, 13, 14, 15] flatten systems into simple, unweighted graphs. They treat all connections identically, failing to distinguish between a message topic, a shared library, and an execution host. Similarly, homogeneous Graph Neural Networks (GNNs) [16, 17, 18] ignore relation-specific message routing.

Currently, no unified framework combines typed multigraph modeling, source-code quality metrics, heterogeneous graph learning, and explainable quality attribution for pre-deployment analysis.

1.3. The Software-as-a-Graph (SaG) Approach

To bridge this gap, we introduce **Software-as-a-Graph (SaG)**, an AI-driven pre-deployment **Static System Analysis (SSA)** framework. SaG ingests Architecture-as-Code manifests and executes a four-stage pipeline:

1. **Typed Multigraph Formulation:** SaG models the distributed architecture as a typed, directed multigraph over five core entity types: Applications, Brokers, Topics, Execution Nodes, and Shared Libraries (Section 3.1).
2. **QoS-Aware Logical Dependency Projection:** Using six formal projection rules, SaG derives a semantic `DEPENDS_ON` dependency layer that captures both sequential cascades (via topics and brokers) and simultaneous blast radii (via shared libraries and node colocation), weighted by declared QoS contracts (Section 3.2).
3. **Explainable Quality Attribution (Diagnostic Pathway):** SaG combines code-level SCA metrics with topological graph properties into a deterministic, hierarchical **Reliability–Maintainability (RM)** quality attribution model (Section 4). Reliability decomposes into **Fault Tolerance** (error propagation depth) and **Availability** (single-point-of-failure exposure). This pathway is a static diagnostic instrument, structurally decoupled from dynamic failure forecasting: it is a linear aggregate of structural and code measurements and models no propagation dynamics by construction, so it explains *why* a component is vulnerable rather than predicting how far a cascade travels. Its standalone rank correlation against simulated impact is correspondingly modest ($\rho \approx 0.19$–0.27, Section 7.3) — a consequence of that scope, not a defect to be tuned away.
4. **Heterogeneous Graph Learning for Failure Forecasting (Predictive Pathway):** To capture the non-linear, multi-hop propagation patterns a deterministic aggregate cannot express, SaG deploys a **Heterogeneous Graph Transformer (HGL)** that uses relation-specific attention and message passing across typed entities to empirically forecast cascading failure blast radii and rank critical components (Section 5).

To ensure methodological rigor, SaG enforces an **input–label independence guarantee**: the learned models and attribution baseline operate strictly on the derived analytical graph G_{analysis}, whereas ground-truth failure impacts are generated by independent discrete-event simulators executing over the raw structural topology $G_{\text{structural}}$ (Section 5.4).

Rationale for Graph Learning vs. Direct Simulation. Because discrete-event simulation $I^*(v)$ defines ground-truth criticality in this study, one might ask: *why train a graph neural network rather than running simulation sweeps directly?* If a complete simulator is available and compute budget is unlimited, simulation alone can identify critical components in an existing system. However, four practical reasons make our hybrid graph learning and attribution framework essential:

1. **Handling Unmeasured Components:** Discrete-event simulators only inject faults into active application processes, leaving passive infrastructure (such as message topics or host nodes) without direct simulation labels (30% to 47% of components per system). The learned GNN generalizes across both labeled and unmeasured entities.
2. **Variance Reduction & Stability:** Cascade simulations are noisy and highly sensitive to random seeds and propagation thresholds (label standard deviation across seeds reaches 0.416). The graph neural network learns a smooth, threshold-marginalized representation that stabilizes predictions across diverse operating regimes.
3. **Sub-Second Speed for CI/CD Quality Gates:** Running exhaustive multi-seed cascade simulations during every code commit or pull request is computationally slow (taking minutes or hours on large meshes). In contrast, trained graph transformers provide inference in under 50 ms, enabling instant CI/CD quality gating.
4. **Diagnostic Explainability:** Simulators and trained GNNs both return impact — a score, a rank, and in the GNN's case an attention distribution showing *where* the model looked (Section 7.3). Neither returns cause attributed in standardised quality terms. A simulator can name precisely which subscriber lost which feed, and is fully inspectable in that sense, but it cannot say whether the component is fragile because it is a single point of failure or because it is a high-coupling maintenance bottleneck — and those two diagnoses call for different remediations (host or broker replication versus topic decoupling and refactoring). Our ISO-grounded RM model supplies exactly that missing layer: once the predictive path has identified a critical component, the diagnostic path says which quality characteristic is at risk and therefore which architectural fix applies.

1.4. Research Questions

Our empirical evaluation investigates four key research questions:

RQ1 (Predictive Efficacy): *How accurately does graph learning predict cascading failure impact and identify critical components compared to traditional, non-learning network metrics?*

RQ2 (Value of Architectural Typing): *Does modeling distinct entity and dependency types (applications, topics, brokers, hosts, and libraries) provide better failure predictions than homogeneous graph models?*

RQ3 (Impact of QoS and Model Sensitivity): *How do middleware Quality-of-Service (QoS) policies, quality weighting calibrations, and simulation thresholds impact prediction accuracy and explainability?*

RQ4 (Real-World Generalization): *How effectively does the framework generalize to real-world, open-source distributed systems across autonomous driving (ROS 2) and cloud-native microservice architectures?*

1.5. Key Contributions

This paper makes four principal contributions:

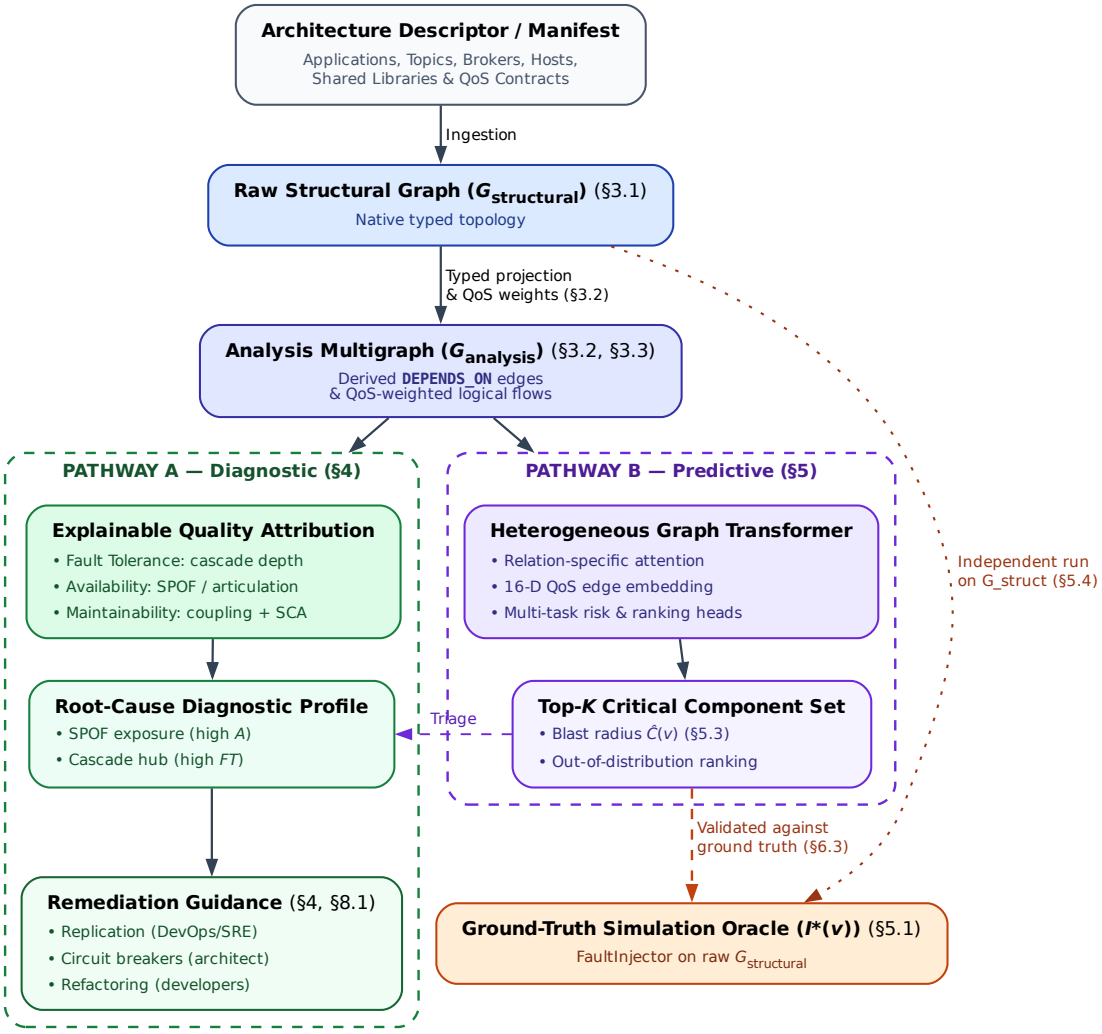

Figure 1: End-to-end architecture of the Software-as-a-Graph (SaG) framework. A shared front end (manifest ingestion → typed multigraph → QoS-weighted `DEPENDS_ON` projection) feeds two deliberately separate pathways. **Pathway A (diagnostic)** produces a standards-grounded quality profile and the remediation it implies; it explains *why* a component is fragile and is not a ranking model. **Pathway B (predictive)** produces a ranked critical set, and is the only pathway validated against the simulation oracle — the oracle scores rankings, which a quality profile is not. The single link between them is triage rather than data flow: the architect applies A's explanation to whatever B flagged. The pathways share no parameters, and the oracle runs on $G_{\text{structural}}$ alone, never on the graph the predictors see (Section 5.4).

1. **A Formal Typed Architecture Model:** A multigraph representation of distributed pub-sub and microservice systems that derives logical dependencies and distinguishes sequential cascade propagation from simultaneous multi-consumer library failures (Section 3).
2. **Heterogeneous Graph Learning for Dependability:** A relation-specific Heterogeneous Graph Transformer (HGL) with 16-dimensional QoS edge feature encoding tailored for pre-deployment failure blast-radius prediction (Section 5).
3. **Standards-Grounded Explainable Attribution:** An interpretable Reliability–Maintainability (RM) quality baseline grounded in ISO/IEC 25010/25019 that bridges code-level SCA metrics with system-level topological criticality (Section 4).
4. **Empirical Evaluation & Real-World Validation:** A rigorous empirical evaluation across seven synthetic topologies (1,545 components) and three authentic real-world systems (Autoware.universe ROS 2, GCP Cloud Microservices, and Train-Ticket; 225 components) under strict input–label independence, establishing the exact conditions where typed graph learning delivers decisive advantages (Sections 6–7).

1.6. Paper Organization

The remainder of this paper is organized as follows: Section 2 reviews related work in distributed systems dependability, static system analysis, and graph representation learning. Section 3 formalizes the Software-as-a-Graph architectural model and dependency projection rules. Section 4 presents the interpretable RM attribution baseline. Section 5 details the Heterogeneous Graph Transformer architecture and simulation ground truth. Section 6 describes the experimental setup, benchmark corpus, and evaluation protocols. Section 7 presents empirical results for RQ1–RQ4. Section 8 discusses architectural implications, sustainability impacts, threats to validity, and concluding remarks.

2. Related Work

This research intersects four foundational domains: distributed systems dependability, static system analysis, graph neural networks for network vulnerability, and software quality models.

2.1. Dependability in Distributed and Pub-Sub Systems

The publish–subscribe (pub-sub) paradigm provides core communication decoupling for scalable distributed software [5]. Modern middleware standards—such as ROS 2 [1], Apache Kafka [2], DDS [3], and MQTT [4]—enable fine-grained Quality-of-Service (QoS) policies governing message durability, transport reliability, and queue deadlines. Prior dependability research in this area has focused primarily on **runtime mechanisms**, such as dynamic consensus protocols, broker clustering, adaptive retransmission, and automated failover.

In parallel, **chaos engineering and runtime verification** [11] inject simulated faults into running staging or production clusters to observe recovery behavior. While runtime fault injection is valuable for testing operational infrastructure, it comes with fundamental limitations:

- It requires fully deployed, operational testbeds.
- It carries operational risk if run near production.
- It operates too late in the software lifecycle to evaluate alternative architectural designs before systems are built.

Our work addresses the complementary, **pre-deployment phase**: predicting systemic cascading vulnerabilities directly from Architecture-as-Code descriptors *before* systems are deployed.

2.2. Static Code Analysis (SCA) vs. Static System Analysis (SSA)

Traditional **Static Code Analysis (SCA)** tools (such as SonarQube) inspect source code Abstract Syntax Trees (ASTs) within individual services. They evaluate cyclomatic complexity [8], class cohesion, module coupling (e.g., LCOM, CBO) [9, 10], and duplicated code to flag internal code smells and defect-prone components [19, 20, 21, 22]. However, SCA is completely blind to runtime topology: it cannot observe inter-service messaging channels, message broker queues, or cross-host container placement.

To close this "Architecture–Code Gap," **Static System Analysis (SSA)** elevates static analysis from single-service source code to the global system architecture. By modeling distributed applications, message topics, brokers, and execution nodes as a connected graph, SSA propagates code-level metrics across architectural dependencies. This allows software teams to catch structural anti-patterns [23, 24, 25, 26] and architectural technical debt [27, 28] early during continuous integration (CI/CD) [29, 30].

2.3. Network Science and Graph Representation Learning

Network science provides established centrality metrics to identify critical nodes, such as degree, closeness, betweenness centrality [12, 14], articulation points, and PageRank [13, 15]. Foundational studies on network robustness [31], cascading overloads [32], and interdependent networks [33] model how failures propagate across connected systems. However, standard network metrics suffer from two major limitations when applied to software architectures:

1. **Dimensional Collapse:** A single centrality number cannot distinguish *why* a component is critical—for instance, whether it is a single point of failure (SPOF), an error-propagating hub, or an over-shared library.
2. **Semantic Collapse:** Standard metrics treat all nodes and edges identically. They conflate fundamentally different architectural entities, such as an asynchronous message topic, a shared C++ library, and a physical execution host.

To move beyond hand-engineered graph metrics, recent research has applied machine learning to network vulnerability (e.g., FINDER [34], DrBC [35], and PowerGraph [36]). However, most existing models use **homogeneous message passing** (GCN [16], GraphSAGE [17], GAT [18]), which averages signals across all connections indiscriminately. Because distributed software architectures are inherently **heterogeneous** (comprising distinct entity types and relationship rules), homogeneous models blur critical architectural boundaries.

Heterogeneous Graph Neural Networks (RGCN [37], HAN [38], HGT [39], MAGNN [40]) solve this by using relation-specific transformations. We build upon the **Heterogeneous Graph Transformer (HGT)** architecture [39] to maintain typed relational semantics when forecasting cascading failure blast radii.

2.4. Software Quality Models and Multi-Criteria Evaluation

Software product quality is standardized by the **ISO/IEC 25010:2023** product quality model [6] (which defines characteristics including Reliability, Maintainability, and Performance Efficiency) and the **ISO/IEC 25019:2023** Quality-in-Use model [7]. Software measurement distinguishes between *internal quality* (measured on static artifacts at rest) and *external quality* (measured on executing software) [41, 42].

Combining multi-attribute structural metrics into an overall quality score is a classic Multi-Criteria Decision Making (MCDM) problem. The **Analytic Hierarchy Process (AHP)** [43] provides a structured pairwise-comparison method with an explicit Consistency Ratio ($CR \leq 0.10$) to ensure weighting schemes remain mathematically sound. In this paper, we use AHP to construct an audited, explainable Reliability–Maintainability (RM) quality baseline, providing transparent architectural diagnostics alongside our learned graph models.

3. The Software-as-a-Graph (SaG) Architectural Model

This section formalizes the Software-as-a-Graph multigraph representation (Section 3.1), the QoS-aware weighting and logical dependency derivation rules (Section 3.2), and the dual graph views utilized throughout the framework (Section 3.3).

Table 1: Comparison of dependability analysis paradigms for distributed systems.

Paradigm	Lifecycle Stage	Topology-Aware	Multi-Typed	Explainable	Zero-Runtime Needed
Static Code Analysis (SCA) [8, 9]	Pre-Deployment	No (Single Service)	No	Yes (Code Smells)	Yes
Chaos Engineering [11]	Post-Deployment	Yes (Live Cluster)	Partial	Partial (Logs)	No (Requires Cluster)
Network Centralities [12, 13]	Pre-Deployment	Yes (Flat Graph)	No	No (Single Scalar)	Yes
Homogeneous GNNs [16, 18]	Pre-Deployment	Yes (Flat Graph)	No	No (Black-Box)	Yes
Software-as-a-Graph (SaG)	**Pre-Deployment**	**Yes (Multigraph)**	**Yes (5 Types)**	**Yes (ISO/IEC RM)**	**Yes (Manifest-Based)**

3.1. Formal Multigraph Definition

We model a complex distributed system as a typed, weighted, directed multigraph:

$$\mathcal{G} = (V, E, \tau_V, \tau_E, w_V, w_E) \tag{1}$$

where:

- V is the set of system entities, partitioned into five disjoint types:

$$V = V_{\text{app}} \cup V_{\text{broker}} \cup V_{\text{topic}} \cup V_{\text{node}} \cup V_{\text{lib}} \tag{2}$$

- E is the set of directed edges connecting entities.
- $\tau_V : V \to \mathcal{T}_V$ and $\tau_E : E \to \mathcal{T}_E$ are typing functions assigning node and edge categories.
- $w_V : V \to [0, 1]$ and $w_E : E \to [0, 1]$ are weighting functions representing entity criticality and coupling strength.

Table 2: Entity and structural edge types in the SaG model.

Entity Type ($\mathcal{T}_V$)	**Architectural Role**	**Concrete System Examples**
Application (V_{app})	Autonomous process producing/consuming messages	ROS 2 node, Kafka microservice, MQTT client
Broker (V_{broker})	Message routing and queuing intermediary	RabbitMQ exchange, Mosquitto, EMQX broker
Topic (V_{topic})	Named logical communication channel	`/sensor/lidar`, `orders.payment.completed`
Node (V_{node})	Physical host or virtualized environment	Bare-metal server, Kubernetes worker, Cloud V
Library (V_{lib})	Shared software package or driver dependency	`librdkafka`, OpenCV, Protobuf runtime
Structural Edge ($\mathcal{T}_E$)	**Direction**	**Semantic Meaning**
`PUBLISHES_TO`	App/Library → Topic	Component publishes messages to topic
`SUBSCRIBES_TO`	App/Library → Topic	Component consumes messages from topic
`ROUTES`	Broker → Topic	Broker manages and routes topic traffic
`RUNS_ON`	App/Broker → Node	Process is hosted on physical/virtual host
`CONNECTS_TO`	Node → Node	Physical network link between hosts
`USES`	App → Library	Application links to shared library dependency

Application and Library entities additionally ingest static code metrics computed via SCA tools (`cm_*` attributes: lines of code, cyclomatic complexity, coupling between objects, LCOM), directly bridging code-level fragility with topological analysis.

3.2. QoS-Aware Weights and Logical Dependency Derivation

In distributed middleware, communication links exhibit varying degrees of coupling based on their Quality-of-Service (QoS) contracts. For example, a `RELIABLE` topic with `TRANSIENT_LOCAL` durability binds communicating services far more tightly than a `BEST_EFFORT` telemetry stream.

Each topic t carries an intrinsic criticality weight $w(t) \in [0, 1]$ combining its declared QoS semantics with two runtime-stress modulators, payload size and publication frequency:

$$w(t) = \beta \cdot \text{QoS}(t) + \alpha \cdot \text{SizeNorm}(t) + \psi \cdot \text{FreqNorm}(t), \quad (\beta, \alpha, \psi) = (0.75,\ 0.15,\ 0.10) \tag{3}$$

where the QoS term is itself an AHP-weighted aggregate of the declared contract,

$$\mathrm{QoS}(t) = w_{\mathrm{rel}} \cdot q_{\mathrm{rel}} + w_{\mathrm{dur}} \cdot q_{\mathrm{dur}} + w_{\mathrm{prio}} \cdot q_{\mathrm{prio}}, \quad (w_{\mathrm{rel}}, w_{\mathrm{dur}}, w_{\mathrm{prio}}) = (0.30,\, 0.40,\, 0.30), \tag{4}$$

with $q_{\mathrm{rel}}, q_{\mathrm{dur}}, q_{\mathrm{prio}} \in [0, 1]$ the normalized reliability, durability and transport-priority scores. Durability carries the highest sub-weight because it dictates message persistence across network partitions; the sub-weight vector is the geometric-mean priority vector of a Saaty pairwise-comparison matrix and is consistent ($CR < 0.05$). The modulators are logarithmically compressed, $\mathrm{SizeNorm}(t) = \log_2(1 + \mathrm{KiB})/50$ and $\mathrm{FreqNorm}(t) = \log_{10}(1 + \mathrm{Hz})/3$, and $w(t)$ is clamped to $[0.01, 1]$ so that best-effort edges remain visible to graph traversals. Every structural communication edge incident on t (`PUBLISHES_TO`, `SUBSCRIBES_TO`, `ROUTES`) inherits $w_E(e) = w(t)$ together with the topic's QoS vector.

The outer split (β, α, ψ) is a declared convex combination rather than an elicited one, and we report its sensitivity directly (Section 7.3.2) rather than defending the particular triple. Because SizeNorm log-compresses realistic payloads into a band roughly 50× narrower than the QoS term's spread, the ordering $w(t)$ induces — the only property any downstream computation consumes — is near-invariant across the entire simplex, including a uniform prior, and downstream rank correlation against the ground-truth oracle moves by at most 0.02 over that whole range.

3.2.1. Logical Dependency Projection (`DEPENDS_ON`)

Structural edges capture explicit deployment connections but omit implicit runtime dependencies. For example, a subscriber depends upon a publisher, yet no direct edge connects them in pub-sub architectures. We derive a single unified semantic relation, `DEPENDS_ON`, directed from *dependent* to *dependency* ("if target fails, source is impacted"):

Table 3: The six `DEPENDS_ON` logical dependency projection rules.

Rule	Dependency Category	Structural Pattern (Dependent → Dependency)	Derived Weight (w)
1	`app_to_app`	Subscriber → Publisher (via shared Topic, incl. transitive `USES`)	$1 - \prod_{t \in T}(1 - w(t))$
2	`app_to_broker`	Publisher/Subscriber → Broker routing its topics	$1 - \prod_{t \in T}(1 - w(t))$
3	`node_to_node`	Host → Host (lifted from inter-host app dependencies)	Lifted $\max w$
4	`node_to_broker`	Host → Broker (lifted from hosted app dependencies)	Lifted $\max w$
5	`app_to_lib`	Application → Shared Library it `USES`	$H(w_V(\mathrm{app}), w_V(\mathrm{lib}))$
6	`broker_to_broker`	Broker ↔ Broker (shared-host fate, symmetric)	$w_V(\mathrm{node})$

Rules 1 and 2 aggregate the several topics T mediating one component pair by *probabilistic union* rather than by a maximum, so that additional parallel failure vectors raise coupling monotonically while preserving $w \in (0, 1]$. Rule 5 uses the harmonic mean $H(x, y) = 2xy/(x + y)$ of the consuming Application's and the shared Library's own vertex weights, which calibrates caller criticality against dependency criticality instead of letting either endpoint dominate. Rules 3 and 4 lift the maximum weight of the component-level dependencies crossing the host boundary.

3.2.2. Sequential Cascades vs. Simultaneous Blasts

A key insight of the SaG model is distinguishing between two fundamentally different failure modes:

- **Sequential Cascade (Rule 1):** When an application publisher fails, downstream subscribers experience data starvation. The failure propagates step-by-step through topics and message queues.
- **Simultaneous Blast (Rule 5):** When a shared software library or execution node crashes, all consuming applications and colocated brokers fail *instantaneously* in a single event.

Retaining entity types and relation-specific dependency rules allows SaG to model both mechanisms, whereas untyped graphs collapse them into identical edges.

On the symmetry of Rule 6. Rule 6 is the one projection rule that is symmetric, and deliberately so: it does not assert that one broker functionally depends on another, but that two brokers co-located on the

same host *share that host's failure domain.* This is the same simultaneous-blast semantics as Rule 5, which is why the derived weight is the shared *Node's* weight rather than either broker's, and why the relation holds in both directions. The mechanism is real in deployed middleware — co-located brokers contend for CPU, page cache, file descriptors and NIC bandwidth, and a host loss takes both at once, which is precisely why operational guidance for Kafka, RabbitMQ and EMQX is to spread brokers across fault domains. What Rule 6 does *not* model is intra-cluster broker coupling proper (partition replication, controller or quorum election, federation and shovel links), which is directional and does not require co-location; extending the schema to express it is left to future work. We also note that the rule is close to inert on our corpus: it fires in four of the eight cached scenarios and contributes twelve directed edges in total across 1,770 components, and because the simulation oracles read only $G_{\text{structural}}$ (Section 5.4), it cannot influence any ground-truth label.

3.3. Dual Graph Views and Architectural Layers

The SaG framework maintains two distinct representations of the system:

1. **Structural Graph ($G_{\text{structural}}$):** The raw graph containing physical and structural edges (`PUBLISHES_TO`, `ROUTES`, `RUNS_ON`, `USES`). This view is consumed exclusively by discrete-event simulators to execute unbiased failure injections (Section 5.1).
2. **Analysis Graph (G_{analysis}):** The projected graph containing derived `DEPENDS_ON` edges annotated with QoS weights and ingested SCA code metrics. All GNN feature representations, graph embeddings, and analytical metrics are computed on G_{analysis}.

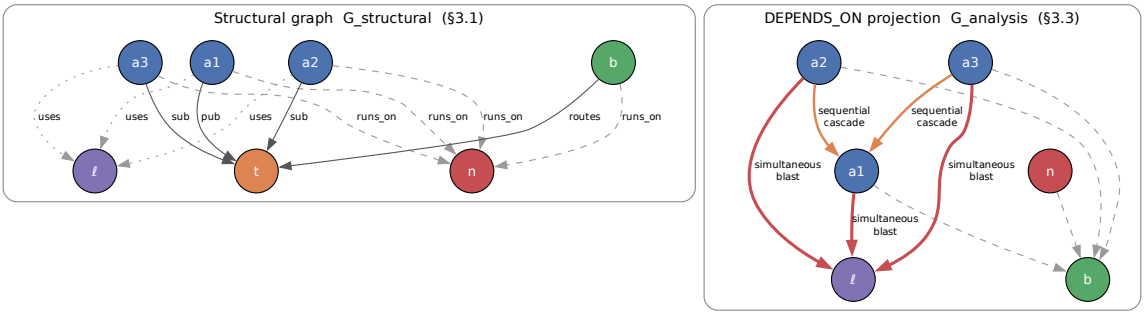

Figure 2: Running example: the raw structural graph (left) and the `DEPENDS_ON` projection derived from it (right). The projection makes implicit runtime dependencies explicit — a subscriber depends on the publishers of its topics even though no structural edge joins them — while the simulators continue to operate on the structural view alone.

G_{analysis} is further organized into four analytical layers (Application, Middleware, Infrastructure, and Global System), allowing architects to evaluate criticality at subsystem levels (e.g., following MIL-STD-498 hierarchical structures).

4. Interpretable Attribution as a Quality Baseline

To provide actionable architectural explanations alongside data-driven predictive rankings, SaG decomposes component and relationship criticality into a multi-dimensional, standards-grounded quality attribution profile.

4.1. Grounding in ISO/IEC Standards

Following **ISO/IEC 25010:2023** (Product Quality Model) [6] and **ISO/IEC 25019:2023** (Quality-in-Use) [7], we formulate two core formal criticality constructs:

- **Component Criticality (D_1):** The degree to which the sudden failure, unexpected termination, or severe degradation of an individual component reduces the system's capacity to deliver required services within its operational context of use.
- **Relationship Criticality (D_2):** The degree of systemic service degradation resulting from the severance, partitioning, or failure of a specific dependency or communication channel while both endpoint components remain operational.

Criticality is evaluated primarily across two orthogonal quality characteristics: **Reliability (R)** and **Maintainability (M)**.

Table 4: The Reliability–Maintainability (RM) quality decomposition.

Dimension	Sub-Characteristic	Architectural Question	Underlying Graph Metrics
Reliability (R)	**Fault Tolerance (FT)**	How broadly does failure propagate?	Reverse PageRank on $G^{\top}$, in-de
	Availability (A)	Is this a single point of failure?	Directed articulation score (raw
Maintainability (M)	**Modularity/Modifiability**	How complex and coupled is this?	Betweenness, QoS-weighted out-

Coverage Scope: SaG focuses on Reliability and Maintainability. Safety (which requires domain-specific hazard logs, e.g., ISO 26262 ASIL ratings) and Security (which requires explicit threat models, e.g., STRIDE) are declared external to purely structural analysis and are left for domain-specific extensions.

4.2. Typed Node Feature Representation

SaG extracts rich feature vectors tailored to the 5 distinct entity types in the software multigraph:

- **Application ($|V_{\mathbf{app}}|$, 23 dims):** Indices 0–17 represent shared topological metrics (in/out degree, betweenness, closeness, reverse PageRank, clustering coefficient, articulation score, bridge load). Indices 18–22 capture source code metrics extracted via Static Code Analysis (SCA): Lines of Code (LOC), Cyclomatic Complexity, Martin's Instability metric ($I_{\text{code}} = \frac{C_e}{C_a+C_e}$), Lack of Cohesion in Methods (LCOM), and composite Code Quality Penalty (CQP).
- **Library ($|V_{\mathbf{lib}}|$, 25 dims):** Same shared topological (0–17) and code quality (18–22) metrics as Application, plus two library-specific extras (indices 23–24): the size of the transitive reverse-`USES` closure (normalized) and the count of distinct subscribers reachable from that closure's published topics (normalized) — the two structural drivers of a library's blast radius under both simulators' cascade rules that the code-quality metrics alone do not capture.
- **Broker ($|V_{\mathbf{broker}}|$, 19 dims):** Indices 0–17 shared topological metrics; index 18 represents normalized queue buffer capacity.
- **Topic ($|V_{\mathbf{topic}}|$, 22 dims):** Indices 0–17 shared topological metrics; indices 18–21 capture publisher count, subscriber count, log message frequency $\log(1 + \text{freq})$, and ordinal QoS criticality.
- **Infrastructure Node ($|V_{\mathbf{node}}|$, 20 dims):** Indices 0–17 shared topological metrics; indices 18–19 capture normalized CPU core allocation and physical memory (RAM).

4.3. Composite Quality Score Formulation

All raw topological and code metrics are rank-normalized to $[0, 1]$. The quality sub-characteristics are formulated hierarchically using the Analytic Hierarchy Process (AHP) [43]:

1. **Fault Tolerance ($FT(v)$):** Measures error cascade potential on the transpose graph $G^{\top}_{\text{analysis}}$ (where edges follow failure propagation from dependency to dependent):

$$FT(v) = 0.45 \cdot \text{RPR}(v) + 0.30 \cdot \text{Deg}_{\text{in}}(v) + 0.25 \cdot \text{CDPot}_{\text{enh}}(v) \tag{5}$$

   where RPR is Reverse PageRank and $\text{CDPot}_{\text{enh}}$ is an enhanced Cascade Depth Potential term.
2. **Availability ($A(v)$):** Identifies structural single points of failure (SPOFs) across five terms — directed articulation severity, its QoS-weighted variant, edge-level irrecoverability, connectivity degradation, and the component's own QoS weight:

$$A(v) = 0.35 \cdot \text{AP}_c^{\text{dir}}(v) + 0.25 \cdot \text{QSPOF}(v) + 0.25 \cdot \text{BR}(v) + 0.10 \cdot \text{CDI}(v) + 0.05 \cdot w(v) \tag{6}$$

3. **Reliability ($R(v)$):** Blends Fault Tolerance and Availability hierarchically:

$$R(v) = \alpha \cdot FT(v) + (1 - \alpha) \cdot A(v), \quad \alpha = 0.36 \tag{7}$$

   Intra-dimension pairwise comparison matrices are audited against Saaty's consistency ratio and measure $CR = 0.001$ (Fault Tolerance), $CR = 0.001$ (Availability), and $CR = 0.000$ (Maintainability) — all well within the $CR \leq 0.10$ acceptability threshold. The shipped intra-dimension weights are a $\lambda = 0.70$ shrinkage blend between the raw AHP-derived vector and a uniform prior (Section 7.3 reports the sensitivity of ranking accuracy to λ).
4. **Maintainability ($M(v)$):** Evaluates structural coupling combined with code-level static analysis across five terms — betweenness, QoS-weighted efferent coupling, the Code Quality Penalty, an afferent/efferent coupling-risk imbalance term, and inverse clustering:

$$M(v) = 0.35 \cdot \text{BT}(v) + 0.30 \cdot w_{\text{out}}(v) + 0.15 \cdot \text{CQP}(v) + 0.12 \cdot \text{CouplingRisk}_{\text{enh}}(v) + 0.08 \cdot (1 - \text{CC}(v)) \tag{8}$$

The baseline composite quality score $Q(v)$ combines both dimensions:

$$Q(v) = 0.80 \cdot R(v) + 0.20 \cdot M(v) \tag{9}$$

When evaluating under a specific ISO/IEC 25019 Context of Use vector $\vec{\omega} = [q_R, q_M]^{\top}$, the score is reweighted dynamically:

$$Q_{\text{domain}}(v) = q_R \cdot R(v) + q_M \cdot M_{\text{static}}(v) \tag{10}$$

Components are categorized into adaptive criticality tiers using box-plot quartile thresholds:

- **CRITICAL:** $Q > Q_3 + 1.5 \cdot \text{IQR}$
- **HIGH:** $Q_3 < Q \leq Q_3 + 1.5 \cdot \text{IQR}$
- **MEDIUM:** $Q_1 < Q \leq Q_3$
- **MINIMAL:** $Q \leq Q_1$

This enables actionable diagnostics: a service scoring high on A but low on FT is diagnosed as a pure SPOF requiring horizontal replication, whereas a service scoring high on FT is an error cascade hub requiring circuit breakers and bulkhead isolation.

5. Graph Learning for Failure-Impact Prediction

While the interpretable RM baseline provides explainable quality profiles, complex non-linear failure interactions and non-local cascade propagations benefit significantly from graph representation learning. This section details our Heterogeneous Graph Transformer (HGT) architecture, 16-dimensional edge representation, multi-task prediction heads, dimension-masked loss formulation, ground-truth simulation oracles, and the strict input–label independence guarantee.

5.1. Ground-Truth Simulation Oracles

To evaluate predictive accuracy prior to deployment without relying on production runtime telemetry, SaG executes discrete-event failure simulations over the raw structural multigraph $G_{\text{structural}}$. We establish a formal taxonomy of three component-level oracles and one relationship-level oracle:

- **Cascade Reachability Oracle ($I^*(v)$):** Implemented via `FaultInjector`, this oracle injects an unexpected node crash at component $v \in V$, propagates cascading failures across dependent topics, brokers, and network links using breadth-first dynamic traversal, and computes the fraction of surviving subscriber feeds severed by the outage. $I^*(v) \in [0, 1]$ serves as the primary continuous target label for training and evaluating GNN predictors.
- **Multi-Metric Composite Oracle ($I_{\mathbf{comp}}(v)$):** Implemented via `FailureSimulator`, this oracle evaluates a multi-faceted failure impact vector:

$$I_{\text{comp}}(v) = 0.35 \cdot \Delta\text{Reachability} + 0.25 \cdot \Delta\text{Fragmentation} + 0.25 \cdot \Delta\text{Throughput} + 0.15 \cdot \Delta\text{FlowDisruption} \tag{11}$$

  $I_{\text{comp}}(v)$ serves as the canonical oracle for architectural quality gate verification.
- **Dynamic Queue-Flow Oracle ($I_{\mathbf{dyn}}(v)$):** Implemented via `MessageFlowSimulator` using the SimPy discrete-event simulation framework [44], this oracle models message emission rates, stochastic network latencies, broker buffer saturation, and dropped delivery counts under fault injection.
- **Relationship (Edge) Removal Oracle ($I_{\mathbf{edge}}(u, v)$):** Evaluates the systemic impact of severing an individual dependency or communication channel while keeping both endpoint components alive:

$$I_{\text{edge}}(u, v) = I_{\text{comp}}(G \setminus \{(u, v)\}) - I_{\text{comp}}(G) \tag{12}$$

Primary oracle declaration. Because the three component-level oracles measure different constructs and agree only partially (below), we designate exactly one of them as primary and hold every ranking claim in this paper to it. **$I^*(v)$ (`FaultInjector`) is the primary oracle** for all predictive-ranking results (Tables 7–9, RQ1–RQ3): it is the only oracle that both supplies training labels and scores held-out predictions, so evaluating a learned ranker against anything else would compare a model to a target it was never fitted to. The remaining oracles have narrower, explicitly named roles: $I_{\text{comp}}(v)$ (`FailureSimulator`) is the Validate-stage oracle for architectural quality gates and anti-pattern detection only, $I_{\text{dyn}}(v)$ (`MessageFlowSimulator`) is used only as an independent, behaviourally-constructed convergent-validity probe, and $I_{\text{edge}}(u, v)$ scores relationships rather than components. Each table and figure names the oracle it was measured against; no number is transferred between them.

These oracles are not interchangeable, and their disagreement bounds what any single one can support. Measured across seven scenarios and five seeds on the Application population, mean Spearman agreement is $\rho = 0.883$ for (I_{dyn}, I^*), $\rho = 0.468$ for (I_{comp}, I^*), and $\rho = 0.465$ for $(I_{\text{comp}}, I_{\text{dyn}})$, with per-scenario minima of 0.756, 0.171, and 0.121 respectively. The behavioural queue-flow oracle and the topological cascade oracle therefore agree strongly on *ordering*, which is genuine convergent evidence: they are constructed differently, one observing message delivery under load and the other reachability over edges, so their agreement is not reducible to a shared construction artifact. The composite oracle is the outlier, agreeing only moderately with either.

Agreement on the *critical set* is markedly weaker than agreement on ordering, and this is the binding limitation. Mean top-K Jaccard overlap at $K = 0.2n$ is 0.42 for the strongest pair and 0.31 for the other two, against 0.111 expected under independent rankings — around three to four times chance, but far from set identity. Two controls establish what this does and does not mean. First, it is not an artifact of tie-breaking: recomputing the overlap so that every component tied with the K-th is admitted changes it by at most 0.005. Second, it is not simply label noise, but neither is it purely construct divergence — the labeler's own seed-to-seed self-agreement spans top-K Jaccard 0.44–1.00 (test–retest $\rho = 0.81$–1.00), so a single oracle compared against itself reaches only 0.44 in its worst scenario. Top-K set identity is an intrinsically unstable

statistic at this K, and the cross-oracle values sit just below the bottom of the range one oracle achieves against itself.

The consequence for how our results should be read is unchanged: a result established against one oracle is not evidence for a claim measured against another, and the instruction to "simply simulate" is under-specified — it does not say which simulator, at which propagation threshold, under which seed, and the answer materially changes the critical set. We report this disagreement as a bound on our own construct validity rather than as corroboration, and we note that we could not reduce it by weighting: applying the same $w(t)$ to both topological oracles does not make them converge (Section 7.3).

5.2. Heterogeneous Graph Transformer Architecture

Because distributed systems comprise heterogeneous entity types (Applications, Libraries, Brokers, Topics, Infrastructure Nodes) and diverse interaction semantics (`PUBLISHES_TO`, `SUBSCRIBES_TO`, `RUNS_ON`, `CONNECTS_TO`, `USES`, etc.), we implement a 3-layer **Heterogeneous Graph Transformer (HGT)** architecture [39] with hidden dimension $D = 64$ and 4 attention heads.

5.2.1. Edge Feature Encoding (16-Dimensional)

To capture continuous QoS constraints and channel semantics, SaG encodes each directed edge $e = (u, v)$ as a 16-dimensional continuous-categorical vector $e_{uv} \in \mathbb{R}^{16}$:

- **Index 0:** Scalar QoS weight $w(e) \in (0, 1]$, computed via the harmonic mean of reliability, durability, priority, deadline, and blocking multipliers.
- **Index 1:** Normalized path count through edge e in G_{analysis}.
- **Indices 2–8:** 7-bit one-hot encoding for edge relationship types (`PUBLISHES_TO`, `SUBSCRIBES_TO`, `ROUTES`, `RUNS_ON`, `CONNECTS_TO`, `USES`, `DEPENDS_ON`).
- **Indices 9–15:** 7 explicit QoS profile dimensions, non-zero only for `PUBLISHES_TO`/`SUBSCRIBES_TO` edges (all other edge types receive zeros): (9) Reliability policy score (0.0 = best-effort, 1.0 = reliable); (10) Durability policy score (0.0 = volatile, 0.5 = transient local, 0.6 = transient, 1.0 = persistent); (11) Normalized message priority (0.0/0.33/0.66/1.0 for low/medium/high/urgent); (12) Deadline constraint active flag (0/1); (13) Log deadline $\log_{10}(1 + \text{deadline\_ns}/10^6)$; (14) Log max blocking time $\log_{10}(1 + \text{max\_blocking\_ms})$; and (15) a QoS heterogeneity flag (1 if the edge's reliability/durability/priority triple deviates from the scenario's modal QoS profile, 0 otherwise).

The `EdgeFeatureEncoder` projects e_{uv} into the hidden space: $e'_{uv} = W_{\text{edge}} e_{uv}$. Prior to relational attention, the edge projection is added directly to the destination node representation: $\tilde{h}_v = h_v + e'_{uv}$.

5.2.2. Type-Specific Projection and Heterogeneous Message Passing

For each source node u and target node v with meta-relation $\tau(e) = (\tau(u), \phi(e), \tau(v))$:

1. **Type-Specific Projection:** Node feature vectors x_v (dimensions 19–23 depending on $\tau(v)$) are mapped into D-dimensional hidden space:

$$h_v^{(0)} = \text{LayerNorm}(\text{GELU}(W_{\tau(v)} x_v)) \tag{13}$$

2. **Relational Mutual Attention:** Query, Key, and Value projections compute relation-specific attention across heads:

$$\text{Attn}(u, e, v) = \underset{\forall u \in \mathcal{N}(v)}{\text{Softmax}} \left(\frac{K(u) W_{\text{att},\phi(e)} Q(\tilde{v})^\top}{\sqrt{D/H}} \right) \tag{14}$$

$$\text{Msg}(u, e, v) = V(u) W_{\text{msg},\phi(e)} \tag{15}$$

3. **Bidirectional Message Passing:** To capture downstream backpressure and upstream cascading failures, message passing is executed over both forward and reverse relations (G_{analysis} and $G^\top_{\text{analysis}}$).

4. **Residual Aggregation and Layer Normalization:**

$$h_v^{(l)} = \text{LayerNorm}\left(h_v^{(l-1)} + \text{Dropout}\left(\sum_{u \in \mathcal{N}(v)} \text{Attn}(u, e, v) \cdot \text{Msg}(u, e, v)\right)\right) \tag{16}$$

5.3. Multi-Task Prediction Heads and Dimension Masking

From the final node embeddings $h_v^{(L)}$, SaG branches into specialized multi-task prediction heads:

- **Reliability Head:** $\hat{R}(v) = \sigma(\text{MLP}_R(h_v)) \in [0, 1]$
- **Maintainability Head:** $\hat{M}(v) = \sigma(\text{MLP}_M(h_v)) \in [0, 1]$
- **Composite Failure Impact Head:** $\hat{I}^*(v) = \sigma(\text{MLP}_C(h_v \parallel \hat{R}(v) \parallel \hat{M}(v))) \in [0, 1]$
- **Relationship Criticality Head:** $\hat{Q}(u, v) = \sigma(\text{TypedEdgeEncoder}_{\phi(e)}(h_u, h_v, e_{uv})) \in [0, 1]$

5.3.1. Dimension-Masked Loss Formulation

The joint optimization objective balances regression accuracy, multi-task dimension learning, ranking fidelity, pairwise ordering, and edge prediction:

$$\mathcal{L} = \mathcal{L}_{\text{composite}} + 0.5 \cdot \mathcal{L}_{\text{dimension}} + 0.3 \cdot \mathcal{L}_{\text{rank}} + 0.1 \cdot \mathcal{L}_{\text{pairwise}} + 0.1 \cdot \mathcal{L}_{\text{consistency}} + 0.3 \cdot \mathcal{L}_{\text{edge}} \tag{17}$$

where $\mathcal{L}_{\text{composite}} = \text{MSE}(\hat{I}^*(v), I^*(v))$, $\mathcal{L}_{\text{rank}}$ is ListMLE ranking loss [45], $\mathcal{L}_{\text{pairwise}}$ is margin-ranking loss, and $\mathcal{L}_{\text{consistency}} = \text{MSE}(\hat{I}^*(v), 0.8\hat{R}(v) + 0.2\hat{M}(v))$.

Dimension Masking: Because dynamic cascade simulation ($I^*(v)$ via `FaultInjector`) observes runtime failure reachability rather than static code maintainability, maintainability ground truth is unobserved during dynamic simulation. We introduce a boolean dimension mask $m = [m_R, m_M] = [1, 0]$:

$$\mathcal{L}_{\text{dimension}} = \frac{1}{\sum_d m_d} \sum_{d \in \{R,M\}} m_d \cdot \text{MSE}(\hat{d}(v), d^*(v)) \tag{18}$$

This prevents the unmeasured maintainability head from being penalized or driven toward zero by backpropagation.

5.3.2. Domain-Reweighted Criticality (Q_{domain})

To ground predictions in ISO/IEC 25019 Context of Use ($\vec{\omega} = [q_R, q_M]^\top$), the composite score is evaluated as:

$$Q_{\text{domain}}(v) = q_R \cdot \hat{R}(v) + q_M \cdot M_{\text{static}}(v) \tag{19}$$

where $M_{\text{static}}(v)$ is drawn directly from the structural analyzer's maintainability baseline (y_{rm} maintainability column), combining learned dynamic reliability with static source-code maintainability. The implementation deliberately refuses to emit $Q_{\text{domain}}(v)$ unless the checkpoint's maintainability head actually received real supervision — under the sole labeler used in every experiment reported here (`FaultInjector`, $m = [1, 0]$), maintainability is unmeasured, so $Q_{\text{domain}}(v)$ is never populated in these results. Domain reweighting sensitivity is instead evaluated directly against the static RM baseline, reported as a dedicated ablation (Section 7.3).

5.4. Input–Label Independence Guarantee

To guarantee experimental rigor and eliminate data leakage, SaG enforces strict architectural decoupling:

- **Feature Space:** Derived exclusively from G_{analysis} (using static structural topology, SCA metrics, and declared QoS contracts).
- **Label Space:** Evaluated exclusively on raw $G_{\text{structural}}$ through independent simulation oracles (`FaultInjector`, `FailureSimulator`, `MessageFlowSimulator`).

No simulation outputs are ever exposed as input features to the GNN or attribution baseline.

6. Experimental Setup

6.1. Datasets and System Corpus

Our evaluation corpus comprises 1,770 components across ten distributed system architectures:

Table 5: Experimental evaluation corpus.

Dataset / Architecture	System Paradigm	$\|V\|$	$\|V_{\text{app}}\|$	Topics	Brokers	Hosts	Libs	$\|E\|$
Autonomous Vehicle (AV)	ROS 2 Cyber-Physical	152	80	40	4	8	20	797
Enterprise Pub-Sub	Kafka Event Mesh	520	300	120	10	40	50	3,245
Financial Trading	Low-Latency Pub-Sub	124	60	35	5	6	18	580
Healthcare Integration	HL7/FHIR Event Mesh	98	50	25	3	8	12	400
Hub-and-Spoke Enterprise	Broker-Centric Messaging	139	70	30	2	12	25	797
IoT Smart City	MQTT Telemetry Mesh	326	200	80	6	30	10	1,322
Microservices Mesh	Cloud-Native Services	186	90	45	6	15	30	680
Autoware.universe [46]	Real-World ROS 2 Autoware	75	32	24	3	6	10	179
Cloud Microservices [47]	Real-World GCP Boutique	60	22	20	4	6	8	128
Train-Ticket [48]	Real-World Microservices	90	41	30	3	8	8	162
Total		**1,770**						

$|V|$ is the sum of all five entity-type counts per scenario and sums to exactly the corpus total stated above. $|E|$ counts every raw structural relationship instance recorded in the scenario file (`PUBLISHES_TO`, `SUBSCRIBES_TO`, `ROUTES`, `RUNS_ON`, `CONNECTS_TO`, `USES`) — the substrate the simulation oracles traverse — not the denser derived `DEPENDS_ON` projection used for GNN training.

The three real-world architectures are hand-transcribed from open-source repositories using dedicated architectural adapters. The seven synthetic scenarios are produced by a parameterised topology generator: each is fully specified by a committed configuration giving a random seed, per-entity-type counts, seven-number summaries (mean, median, standard deviation, min, max, Q_1, Q_3) for application publish and subscribe fan-out, for applications per host, for library fan-in and for topic payload size, and categorical distributions over the three QoS dimensions. Degree distribution and clustering are *emergent* from those inputs rather than specified directly. Table 6 gives the parameters that determine each topology's shape; the complete configurations are in the replication package.

Table 6: Generative parameters of the seven synthetic evaluation scenarios, read directly from the committed configurations. Counts are Applications/Topics/Brokers/Hosts/Libraries. Fan-out figures are per-application means over the configured distribution; the modal QoS column gives the most common reliability/durability/priority value and the range of topic shares carrying them.

Scenario	Config	Seed	Counts	Pub	Sub	Modal QoS (
Autonomous Vehicle (AV)	`scenario_01_autonomous_vehicle`	1001	80/40/4/8/20	2.5	5.0	RELIABLE/T
Enterprise Pub-Sub	`scenario_07_enterprise_xlarge`	7007	300/120/10/40/50	3.0	4.5	RELIABLE/T
Financial Trading	`scenario_03_financial_trading`	3003	60/35/5/6/18	4.0	6.0	RELIABLE/P
Healthcare Integration	`scenario_04_healthcare`	4004	50/25/3/8/12	2.5	3.0	RELIABLE/P
Hub-and-Spoke	`scenario_05_hub_and_spoke`	5005	70/30/2/12/25	2.0	7.0	RELIABLE/T
IoT Smart City	`scenario_02_iot_smart_city`	2002	200/80/6/30/10	2.0	1.5	BEST_EFFO
Microservices Mesh	`scenario_06_microservices`	6006	90/45/6/15/30	1.5	2.0	RELIABLE/T

6.1.1. Reproducibility of the Corpus

The corpus is regenerable rather than merely archived. Each dataset is produced from its configuration by

```
python cli/generate_graph.py batch -input-dir data/scenarios -output-dir <dir>
```

and a manifest records, per dataset, the seed, the entity counts, the generating commit, and a SHA-256 digest of the emitted topology. A regression test asserts that every committed dataset regenerates *byte-identically* from its configuration and that the manifest matches what is on disk, so a divergence between the published numbers and the published data fails the test suite rather than passing unnoticed. A third party can therefore reproduce the exact graphs these results were computed on, not merely graphs drawn from the same distribution.

6.2. Baselines and Evaluated Predictors

We compare five primary predictor configurations:

1. **Topo-BL:** Unweighted structural centrality (betweenness centrality and articulation point scoring).
2. **Topo-QoS:** QoS-weighted topological centrality baseline.
3. **RM / $Q(v)$:** Deterministic hierarchical quality attribution baseline (Section 4).
4. **GL / GL-QoS:** Homogeneous Graph Attention Networks (GAT) [18] trained on the flattened, untyped graph projection.
5. **HGL / HGL-QoS:** Relation-specific Heterogeneous Graph Transformers (Section 5).

6.3. Evaluation Metrics and Protocols

- **Ranking Accuracy:** Spearman rank correlation (ρ) and Kendall's tau (τ) between predicted rankings and ground-truth simulated impact $I^*(v)$, the primary oracle declared in Section 5.1.
- **Critical-Set Identification:** $F_1@K$, Precision@K, and Recall@K for top-K critical components, where K is 20% of the evaluated node population, rounded to nearest. Because the predicted and true sets both contain exactly K elements, precision, recall and F_1 coincide at K; the figure is a top-K overlap and we read it as such.
- **Statistical Significance:** Paired Wilcoxon signed-rank tests [49] ($p < 0.05$) and bootstrap 95% confidence intervals ($B = 2,000$) [50, 51].

6.3.1. Evaluation Population

Every predictor in a given table is scored on an identical node set, resolved from the graph and the labels alone — never from any variant's predictions — so that no comparison is confounded by which nodes a particular method happened to score. Unless stated otherwise that set is the **Application** population: it is the population the framework's central claim is about (topology predicts application-layer cascade criticality), and it is the only one every variant can score. This matters more than it may appear. Pooling node types into a single correlation mixes populations with different impact scales and base rates, and on this corpus that pooling is not benign — it moves the RM composite's rank correlation *outside* the range spanned by its own per-type correlations (Section 7.3). We therefore report stratified, single-population figures throughout, and flag explicitly wherever a pooled figure appears.

6.3.2. Evaluation Protocols

- **In-Distribution Evaluation:** 60% train / 20% validation / 20% test node splits pinned by node identity within each scenario, evaluated over five random seeds $\{42, 123, 456, 789, 2024\}$.
- **Inductive Leave-One-Scenario-Out (LOSO):** Across all eight cached scenarios (the seven synthetic scenarios plus the ATM case study, Section 7.5), models are trained on the remaining seven and evaluated zero-shot on the held-out scenario, for eight folds total, to test out-of-distribution generalizability.
- **Real-World Architectural Transfer:** Evaluating zero-shot transfer on authentic open-source architectures.

Table 7: In-distribution held-out Spearman rank correlation (ρ) against $I^*(v)$ (seed means over 5 seeds with bootstrap 95% CIs; n is held-out Application count).

Scenario	n	Topo-BL	Topo-QoS	GL	GL-QoS	HGL	HGL-QoS
AV System	16	0.283	0.701	**0.790**	0.689	0.789	0.649
Enterprise	60	0.420	0.789	0.875	0.459	0.880	**0.891**
Financial Trading	12	0.289	0.700	0.672	0.536	**0.854**	0.770
Healthcare	10	−0.101	**0.772**	0.590	0.463	0.652	0.645
Hub-and-Spoke	14	0.234	0.359	**0.554**	0.052	0.534	0.296
IoT Smart City	40	−0.063	0.073	0.238	0.259	**0.881**	0.842
Microservices	18	0.401	**0.707**	0.564	0.571	0.483	0.476
Mean	—	**0.209**	**0.586**	**0.612**	**0.433**	**0.725**	**0.653**

Table 8: Paired Wilcoxon signed-rank tests across scenarios ($n = 7$, two-sided).

Comparison	$\Delta\rho$	Won	Wilcoxon W	p-value	Significance
HGL vs. Topo-BL	**+0.516**	7/7	0.0	**0.0156**	**Statistically Significant** ($p < 0.05$)
HGL vs. GL-QoS	**+0.292**	6/7	1.0	**0.0312**	**Statistically Significant** ($p < 0.05$)
HGL vs. Topo-QoS	**+0.139**	5/7	9.0	0.469	Not significant
GL vs. Topo-QoS	**+0.026**	4/7	11.0	0.688	Not significant
HGL vs. GL	**+0.113**	4/7	9.0	0.469	Not significant
HGL-QoS vs. HGL	**−0.072**	1/7	3.0	0.078	Not significant (marginal; HGL-QoS trails in-distribution)

7. Results and Empirical Analysis

7.1. RQ1: Graph Learning vs. Structural Baselines

Table 7 presents the in-distribution held-out Spearman rank correlation (ρ) against simulated cascade impact $I^*(v)$ across all seven synthetic scenarios.

7.1.1. Out-of-Distribution (LOSO) Generalization

Under inductive Leave-One-Scenario-Out (LOSO) cross-validation, the model must predict cascading criticality on an unseen system topology:

Table 9: Inductive Leave-One-Scenario-Out (LOSO) evaluation, Application population, eight folds. The RM baseline is included as a reference point rather than as a competing predictor: it is the diagnostic path of Section 1.2 and is not proposed as a ranking model (Section 4.1). Its row quantifies how much the predictive path adds over static attribution, which is the only sense in which the comparison is meaningful.

Model Variant	Mean LOSO ρ	Std ρ	Critical-Set $F_1@K$	Requires Training
Topo-BL	0.301	0.126	0.363	No
Topo-QoS	0.571	0.181	0.380	No
RM / $Q(v)$ Baseline	0.190	0.131	0.328	No
GL (Homogeneous)	0.086	0.123	0.237	Yes
GL-QoS (Homogeneous)	0.363	**0.089**	0.339	Yes
HGL (Typed Heterogeneous)	0.451	0.149	0.341	Yes
HGL-QoS (Typed + QoS)	**0.608**	0.144	**0.414**	Yes

Eight LOSO folds are reported (the seven synthetic scenarios plus the ATM case study of Section 7.5), each holding out one scenario and training on the remaining seven. Every variant is scored on the identical Application node set per fold (Section 6.3); paired Wilcoxon tests below are over the eight folds.

Key Insights for RQ1:

1. **The learned models beat the learned baselines decisively; the untrained QoS baseline is the real competitor.** HGL-QoS is the best configuration overall ($\rho = 0.608$, $F_1@K = 0.414$), and its margin over both homogeneous GNNs is large and significant (+0.245 over GL-QoS, 8/8 folds, $p = 0.0078$). Against *Topo-QoS*, however — a training-free QoS-weighted centrality baseline — the margin is +0.037, won in only 5 of 8 folds, and is **not statistically significant** ($p = 0.74$). We state this plainly: on out-of-distribution Application ranking, we cannot claim that heterogeneous graph learning outperforms a well-constructed structural baseline, only that it matches it while additionally supplying the typed attention and multi-task outputs the baseline cannot.
2. **Critical-set detection is where the learned model separates.** HGL-QoS improves $F_1@K$ over Topo-QoS by +0.034 (0.414 vs. 0.380, a 8.9% relative gain) and over GL by +0.177. The advantage is real but far smaller than a pooled-population reading of the same experiment suggests.
3. **QoS encoding is what carries the typed model out of distribution.** HGL-QoS leads HGL by $\Delta\rho = +0.157$, winning 7 of 8 folds ($p = 0.0156$) — a larger and more consistent effect than the in-distribution comparison in Table 8 suggests in the opposite direction. See Section 7.3.
4. **The RM baseline is weakly predictive, not anti-predictive.** $\rho = 0.190$ under distribution shift: better than the untyped GL model and worse than every QoS-aware variant. RM's value lies in interpretable attribution rather than standalone ranking, but it is not noise.

A note on population. An earlier version of this table pooled every node type carrying a simulated label. That pooling inverted several conclusions — it reported the RM baseline at $\rho = -0.014$ rather than +0.190, and GL at 0.381 rather than 0.086, because GL's pooled score was carried almost entirely by Broker nodes whose impact distribution differs in both scale and base rate from Applications'. This is the Simpson's-paradox hazard documented in Section 8.2, and it is why every figure in this paper is now reported on a single, stated population.

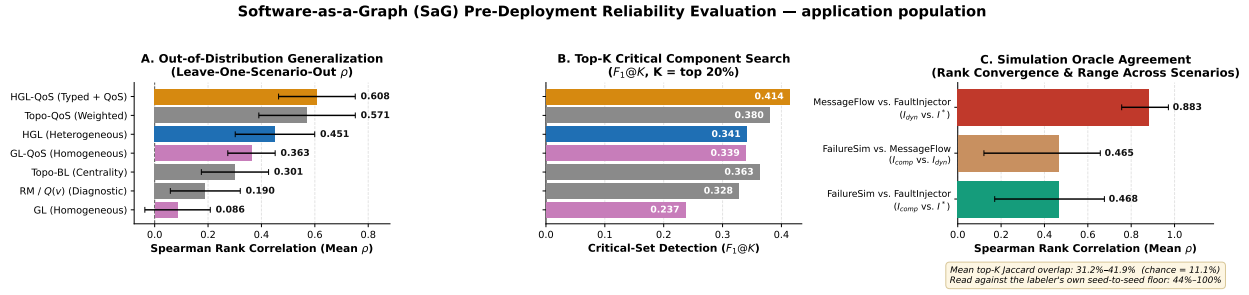

Figure 3: Results at a glance, all on the Application population. **(A)** Out-of-distribution rank correlation per variant, whiskers showing σ across the eight LOSO folds; note that the training-free Topo-QoS baseline places second. **(B)** Critical-set detection at $K = 20\%$. **(C)** Agreement between the three simulation oracles, whiskers showing the observed range across scenarios; the annotation gives top-K set agreement against both chance and the labeler's own seed-to-seed floor.

7.2. RQ2: Value of Typed Heterogeneity

To evaluate the contribution of node and edge typing, we compare relation-specific HGL against homogeneous GL across different validation regimes:

- **In-Distribution:** Heterogeneous HGL leads homogeneous GL by $\Delta\rho = +0.113$ (0.725 vs. 0.612; not statistically significant, $p = 0.469$, 4/7 scenarios won). On familiar topologies, homogeneous GNNs can partially approximate type distinctions through structural degree signatures, so the typed model holds only a point-estimate edge. The contrast with the out-of-distribution result below is the finding: typing buys little when the topology is already known and a great deal when it is not.

- **Out-of-Distribution (LOSO):** This is where typing decides the outcome. Heterogeneous HGL outperforms homogeneous GL by +0.365 ($\rho = 0.451$ vs. 0.086), winning *all eight* folds (paired Wilcoxon,

$p = 0.0078$), and the gap between the QoS-aware variants is +0.245 (HGL-QoS 0.608 vs. GL-QoS 0.363), also 8/8 and $p = 0.0078$. The untyped model very nearly fails to transfer at all on Application ranking: at $\rho = 0.086$ it is below even the unweighted structural baseline (0.301). When encountering unseen topologies, relation-specific message passing is not an incremental refinement but the difference between a model that generalizes and one that does not.

7.2.1. Empirical Edge-Removal Analysis

We further probed edge criticality by simulating the removal of candidate relationship channels while keeping both endpoint components alive. On the `av_system` topology across 50 candidate bridge edges, 46 edges exhibited zero downstream cascade impact, while the 4 edges with non-zero impact were direct library communication channels (`PUBLISHES_TO` / `SUBSCRIBES_TO`). This demonstrates that individual communication links are largely redundant, whereas component-level failures and shared library dependencies induce disproportionate systemic disruption.

7.3. RQ3: Ablations and Sensitivity Analysis

7.3.1. QoS Feature Encoding

Explicitly encoding continuous QoS attributes in the GNN (HGL-QoS) does not yield a uniform effect — it trades a little in-distribution accuracy for a large out-of-distribution gain. The two directions are summarised below; note that they are measured under different protocols on different fold counts, though both are scored on the Application population.

Table 10: HGL-QoS against HGL under both protocols. Both are Application-scored; the regimes differ in fold count (7 in-distribution scenarios vs. 8 LOSO folds, the latter including the ATM case study).

Protocol	**HGL**	**HGL-QoS**	$\Delta\rho$	**Folds won**	**Wilcoxon p**
In-distribution (Table 7)	0.725	0.653	−0.072	1/7	0.078 (n.s.)
Inductive LOSO (Table 9)	0.451	**0.608**	**+0.157**	**7/8**	**0.016**

The apparent contradiction between the two tables is a genuine regime effect, not an inconsistency. In-distribution, HGL-QoS *trails* the base typed model, but the deficit is not statistically significant and is small relative to its own scatter (across-scenario $\sigma = 0.35$ for HGL-QoS against a 0.072 gap); it is concentrated in two structurally atypical topologies, Hub-and-Spoke (−0.238) and AV (−0.140), with the remaining five within ±0.084. This is consistent with QoS features adding redundant signal, and mild overfitting risk, on topologies the model has already seen — the derived `DEPENDS_ON` graph already embeds much of the same routing and coupling information through its edge weights.

Under LOSO the relationship reverses decisively and *is* significant: HGL-QoS gains +0.157 and wins 7 of 8 folds. When the topology itself is unseen, structural degree signatures no longer transfer, whereas a declared QoS contract means the same thing in an unfamiliar architecture as in a familiar one — `RELIABLE` and `PERSISTENT` carry their semantics across deployments in a way that a betweenness percentile does not. We therefore read QoS encoding as *situational* rather than redundant: insurance against distribution shift, bought at a small and statistically insignificant in-distribution cost. For a practitioner the rule is simple — use HGL when the target system resembles the training corpus, HGL-QoS when it does not, and HGL-QoS by default when that is unknown.

7.3.2. Topic-Weight Coefficients (β, α, ψ)

The outer split of Equation 3 is a declared convex combination, not an elicited one, so we measure what rests on it rather than argue for the particular triple. We swept (β, α, ψ) over a grid spanning the whole simplex — including the QoS-only corner $(1, 0, 0)$ and a uniform prior $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ — and propagated each point through the derived `DEPENDS_ON` edge weights into two closed-form scorers.

The coefficients are not load-bearing. Across the entire simplex the induced ordering of $w(t)$ never falls below $\rho = 0.925$ against the shipped ordering, and downstream rank correlation moves by at most 0.020 (Topo-QoS)

Table 11: Sensitivity of the topic-weight ordering and of downstream rank correlation against $I^*(v)$ to the declared coefficients (β, α, ψ). Application population, mean over seven scenarios; 375 topics.

(β, α, ψ)	ρ **of** $w(t)$ **vs. shipped**	**Topo-QoS** ρ	**RM** ρ
$(0.75, 0.15, 0.10)$ *shipped*	1.000	0.599	0.268
$(0.90, 0.05, 0.05)$	0.9999	0.597	0.269
$(0.85, 0.15, 0.00)$	0.936	0.607	0.270
$(1.00, 0.00, 0.00)$	0.925	0.617	0.258
$(0.60, 0.20, 0.20)$	0.995	0.602	0.268
$(0.50, 0.25, 0.25)$	0.986	0.602	0.266
$(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ *uniform*	0.925	0.603	0.266
Spread across grid	—	**0.020**	**0.012**

and 0.012 (RM) — an order of magnitude below the differences between the predictor families the tables above compare. The mechanism is visible in the terms themselves: over the 375 corpus topics the QoS term contributes a weighted standard deviation of 0.188, the frequency term 0.021, and the payload term 0.0039, because SizeNorm log-compresses realistic payloads into a band roughly 50× narrower than the QoS term's spread. The payload term is therefore close to inert on this corpus whatever α is set to, which we report as a limitation of the construct rather than as a tuned parameter. We emphasise the direction of this claim: it establishes that the declared triple does not drive any reported result, *not* that $(0.75, 0.15, 0.10)$ is optimal — the QoS-only corner is in fact marginally better for Topo-QoS (0.617 against 0.599) and marginally worse for RM. The inner QoS split $(0.30, 0.40, 0.30)$ is a separate matter: it is AHP-derived, its Saaty matrix is consistency-audited, and both are pinned by regression tests.

7.3.3. Intra-Dimension AHP Weight Shrinkage

We evaluated the sensitivity of the RM attribution baseline by sweeping a shrinkage parameter $\lambda \in [0, 1]$ blending internal term weights from uniform prior ($\lambda = 0$) to calibrated AHP judgement ($\lambda = 1$). Note that λ shrinks only the *intra-dimension* vectors: the composite weights $(q_R, q_M) = (0.80, 0.20)$ are declared constants and are identical at every λ.

Table 12: Sensitivity of RM rank correlation against $I^*(v)$ to the AHP shrinkage parameter λ, Application population, mean over seven scenarios.

λ **Setting**	**0.00 (Uniform)**	**0.50**	**0.70 (Default)**	**0.80**	**1.00 (Raw AHP)**
Mean Rank Correlation (ρ)	**0.347**	0.291	0.268	0.256	0.234

The AHP judgement does not improve ranking, and we report that directly. As Figure 4 shows, ρ declines monotonically as the weights move from the uniform prior toward the elicited judgement. The uniform prior is the best setting in the sweep ($\rho = 0.347$ against 0.234 at raw AHP), it wins in all seven scenarios individually (paired Wilcoxon, $p = 0.0156$), and the shipped $\lambda = 0.70$ default costs $\Delta\rho = -0.079$ relative to it ($p = 0.0156$). There is no plateau around the default. The honest reading is that the elicited intra-dimension hierarchy is a *transparency* device — it makes the composite auditable and its terms nameable to an architect — and not a ranking-accuracy device; on this corpus, weighting the terms equally ranks components better. We retain $\lambda = 0.70$ because RM's role is interpretable attribution rather than standalone ranking (Section 4.1), but we do not claim the sweep validates the expert hierarchy, because it does not.

A measurement correction. An earlier version of this analysis scored $Q(v)$ from features restricted to the Application–Library `DEPENDS_ON` projection — the feature substrate built for GNN training — rather than from the full typed graph. On that projection no Application is an articulation point and no incident edge is a bridge in six of the eight scenarios, so four of Availability's five terms vanish and $A(v)$, which carries $w_R(1 - \alpha) \approx 0.51$ of the composite, is constant across the Application population. Sweeping λ against that variant measured a degenerate score and returned uniformly negative ρ (from -0.146 at $\lambda = 0$ to -0.112

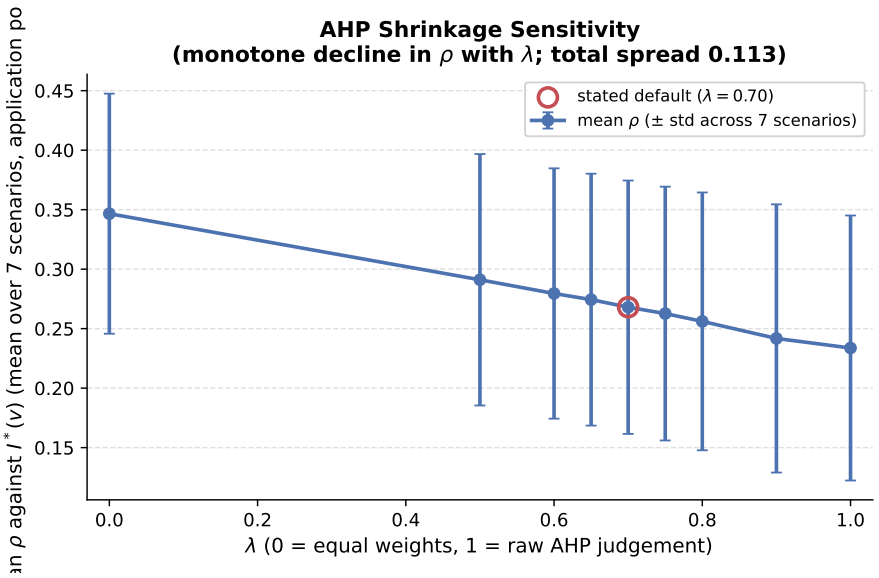

Figure 4: Sensitivity of the RM composite's rank correlation against $I^*(v)$ to the AHP shrinkage parameter λ, Application population, mean over seven scenarios. ρ declines monotonically toward the elicited judgement: the uniform prior at $\lambda = 0$ is the best setting in the sweep.

at $\lambda = 1$). The figures above are computed through the full analysis pipeline, which is the RM baseline defined in Section 4.3; both series are retained in the replication artifact. The same correction applies to the threshold and domain-weighting ablations below, which shared that scorer.

7.3.4. Convergent Validity Across Simulation Oracles

To test construct validity, we compared node criticality rankings across our three simulation engines — the topological cascade reachability oracle $I^*(v)$ (`FaultInjector`), the multi-metric composite oracle $I_{\text{comp}}(v)$ (`FailureSimulator`), and the discrete-event queue-flow simulator $I_{\text{dyn}}(v)$ (`MessageFlowSimulator`) — over seven synthetic scenarios and five seeds, on the Application population.

Table 13: Inter-oracle agreement. Chance-level top-K Jaccard at $K = 0.2n$ is 0.111; the tie-robust column admits every component tied with the K-th.

Oracle pair	Mean ρ (range)	Mean τ	Jaccard@K	Tie-robust
I_{dyn} vs. I^*	**0.883** (0.756–0.972)	0.745	0.424	0.419
I_{comp} vs. I^*	0.468 (0.171–0.677)	0.349	0.307	0.312
I_{comp} vs. I_{dyn}	0.465 (0.121–0.658)	0.348	0.313	0.313

Ordering converges; the critical set does not. The behavioural queue simulator and the topological cascade oracle agree strongly on ordering ($\rho = 0.883$, never below 0.756 on any scenario). Because the two are constructed on different principles — one observing delivered message rates under load, the other reachability over edges — this agreement is not reducible to a shared construction artifact, and it is the strongest convergent evidence available to us that $I^*(v)$ tracks dynamic service disruption rather than only graph topology. The composite oracle is the outlier, agreeing only moderately with either ($\rho \approx 0.47$).

Set-level agreement is a different and weaker story, and we report it as the binding limitation rather than burying it. Top-K Jaccard is 0.42 for the strongest pair and 0.31 for the others, against 0.111 expected under independent rankings. We ran two controls before interpreting this. It is *not* a tie-breaking artifact: admitting every component tied with the K-th moves the figure by at most 0.005. And it is not simply label noise, though noise is a substantial part of it — the labeler's own seed-to-seed self-agreement spans top-K

Jaccard 0.44–1.00, so one oracle compared against *itself* reaches only 0.44 in its worst scenario. Top-K set identity at $K = 0.2n$ is therefore an intrinsically unstable statistic, and the cross-oracle values sit just below the floor a single oracle achieves against its own reruns. The defensible claim is about ranking, not about membership of a fixed-size critical set.

A falsified hypothesis, reported. We tested whether weighting both topological oracles by the same $w(t)$ makes them converge — the mechanism by which QoS weighting would be a construct-validity improvement rather than merely a modelling choice. It does not: $\rho(I_{\text{comp}}, I^*)$ is 0.468 with QoS weighting and 0.477 without it ($\Delta\rho = -0.009$; Jaccard 0.307 against 0.306). Shared weighting has no measurable effect on inter-oracle agreement in either direction, and we record the negative result rather than leaving the artifact uncited.

7.3.5. Domain-Specific Weighting Sensitivity

We investigated the impact of Context of Use reweighting $\vec{\omega} = [q_R, q_M]^\top$ across 10 evaluation scenarios by sweeping the composite reliability weight w_R over its full range and reading off three points on that one curve: the shipped static default ($w_R = 0.80$), an unweighted equal split ($w_R = 0.50$), and the domain-derived value per scenario. The three are statistically indistinguishable: mean $\rho = 0.345$ (static), 0.342 (equal), 0.345 (domain-derived), so domain derivation beats equal weighting by $\Delta\rho = +0.002$ and trails the static default by $\Delta\rho = -0.001$. Neither difference is meaningful, and the sweep explains why: over the *entire* $w_R \in [0, 1]$ range mean ρ moves only from 0.341 to 0.354, a total spread of 0.014. The composite weight is simply not a lever on ranking. Two structural facts compound this: across all ten declared domains the derived w_R lies in $[0.70, 0.80]$, i.e. at most 0.10 from the static default, and mean Kendall rank fidelity between domain-derived and static rankings is $\tau = 0.976$ — the two weightings order components almost identically. We therefore read this ablation as confirming the scoping in Section 4.1: operational Context of Use reweighting is an *attributional* mechanism — explaining why a component matters in stakeholder terms — and not a ranking-improvement device. It should not be reported as either an accuracy gain or an accuracy loss.

7.3.6. Threshold and Normalization Sensitivity

We swept 7 scenarios × a cascade propagation-threshold parameter $\in \{0.0, 0.1, 0.2, 0.35, 0.5, 0.75, 1.0\}$ controlling the ground-truth oracle's eligibility cutoff for cascade propagation, and separately swept 3 label-normalization techniques (robust, min-max, standard) holding the threshold fixed. Ranking performance is more sensitive to the propagation threshold ($\Delta\rho = 0.099$ across the extreme settings; mean ρ rises from 0.183 at threshold 0 to 0.281 at threshold 0.5 and plateaus thereafter, the shipped 0.35 sitting inside that plateau at 0.276) than to normalization choice ($\Delta\rho = 0.027$; robust 0.268 against 0.241 for both min-max and standard). The threshold is thus the more consequential free parameter of the two, and the shipped setting is on the flat part of its curve rather than at a tuned peak. A small spread under either sweep means the reported ordering is stable under that parameter — it is not, by itself, evidence that the ordering is *correct*.

7.3.7. Anti-Pattern Detection and CI/CD Quality Gates

Validating our rule-based architectural anti-pattern catalog against the multi-metric composite oracle $I_{\text{comp}}(v)$ yielded a mean detection $F_1 = 0.3781$ across 8 system scenarios — but F_1 alone understates what the catalog does: mean precision is 0.239 against mean recall 0.900, meaning the catalog flags 94.2% of all components on average and trades precision for near-exhaustive coverage of true positives. This is a deliberate high-recall CI/CD posture (a missed critical component is more costly than a false positive requiring manual triage) rather than an accuracy shortfall, but should be read as such rather than via F_1 in isolation. One detector, `DEEP_PIPELINE`, is excluded from this evaluation: it enumerates every simple source-to-sink path and does not terminate within ten minutes on the 50-application Healthcare topology — the blowup is itself a reportable scalability limit of exhaustive path enumeration, not a measurement gap. Analysis execution times for the remaining 18 detectors ranged from 0.04 s (small scenarios, 50 nodes) to 54.85 s (enterprise event meshes, 300 nodes), confirming that SaG runs comfortably within standard pre-commit and pull-request CI/CD quality gate budgets.

We additionally stratify the RM composite's rank correlation by node type: $\rho = 0.503$ (Application), 0.395 (Broker), 0.142 (Node), while the *pooled* correlation across all types is $\rho = 0.028$ — outside the per-type range entirely. This is a Simpson's-paradox effect (aggregating across populations with different scales/base rates reverses the within-group trend), and it means the pooled figure is not a summary of the per-type figures: only the stratified, per-type numbers should be quoted when discussing RM's structural predictive power.

7.3.8. Using RM Correctly: Score Within a Node Type

The stratification above is not only a reporting convention for this paper; it is a usage rule for the framework, and getting it wrong changes which components an architect is told to fix. We therefore state it operationally.

The rule. Score, tier and rank components *within* a node type. Never derive a single ranking, or a single criticality threshold, from a population that mixes Applications, Brokers, Topics, Infrastructure Nodes and Libraries. Their criticality scores differ in scale and in base rate, so a shared threshold measures type membership as much as it measures risk.

The mechanism is already in the framework. The layer projections of Section 3.3 are the stratification device: π_{infra} reports Nodes only, π_{mw} reports Brokers only, and π_{app} reports Applications and Libraries. Only π_{system} spans all five types. Practitioners should read π_{system} as a structural and visualisation view of the whole architecture, and take ranking or gating decisions from the single-type layers.

What pooling costs, measured. We classified every component in the eight-scenario corpus twice — once against one box-plot over the whole system layer, once within its own node type — and compared the resulting CRITICAL/HIGH/MEDIUM/LOW/MINIMAL tiers. Across 1,619 components, **62.8% land in a different tier**, and **19.0% cross the CRITICAL/HIGH boundary** that determines whether a component is surfaced for action at all. The effect is stable across scenarios (tier changes 51–69%, boundary crossings 14–22%), so it is a property of mixing the populations rather than of any one topology. The direction is systematic: pooling suppresses the top of whichever type carries the lower score scale, because the higher-scaled type occupies the upper quartile outright.

What we changed. The classifier now derives quartiles and fences within each node type, and the per-dimension criticality flags use the same fence that produced the component's tier, so flag and tier cannot disagree. Types with too few members for stable quartiles fall back to the pooled fence rather than being scored on a handful of samples. The residual case is π_{app}, which still pools Applications with Libraries; their LOSO correlations overlap (0.190 against 0.105), so we retain the pairing and note it as a scope condition rather than a defect.

7.3.9. HGT Attention Weight Analysis

Extracting relational attention matrices from the trained HGT model on the ATM Case Study (Figure 5) revealed that the model places its highest single attention weight ($\alpha_{uv} = 1.00$) on a `USES` edge (an Application's dependency on a shared Library), with the next tier ($\alpha_{uv} \approx 0.50$) split between a `ROUTES` edge (Broker $\rightarrow$ Topic) and further `USES`/`SUBSCRIBES_TO` edges. This pattern — dominant attention on library-dependency and broker-routing channels rather than direct publish/subscribe message flow — reflects the shared-library blast-radius and broker-centrality failure pathways identified elsewhere in this study (Section 7.2's edge-removal analysis, Section 3's simultaneous-failure semantics for shared libraries) rather than sequential pub-sub cascade propagation.

7.4. Scalability and Computational Cost

A learned model invites the question of whether it remains usable as systems grow. On this pipeline the answer is that the neural network is not the constraint — the deterministic structural analysis that feeds it is, by more than two orders of magnitude.

The GNN is the cheapest stage. Inference on a 2,000-component system takes 44 ms — a handful of sparse matrix products whose cost grows close to linearly in $|E|$ — while the structural analysis preceding it takes 24 s. The ratio widens with scale rather than narrowing, from 12× at 249 components to 545× at

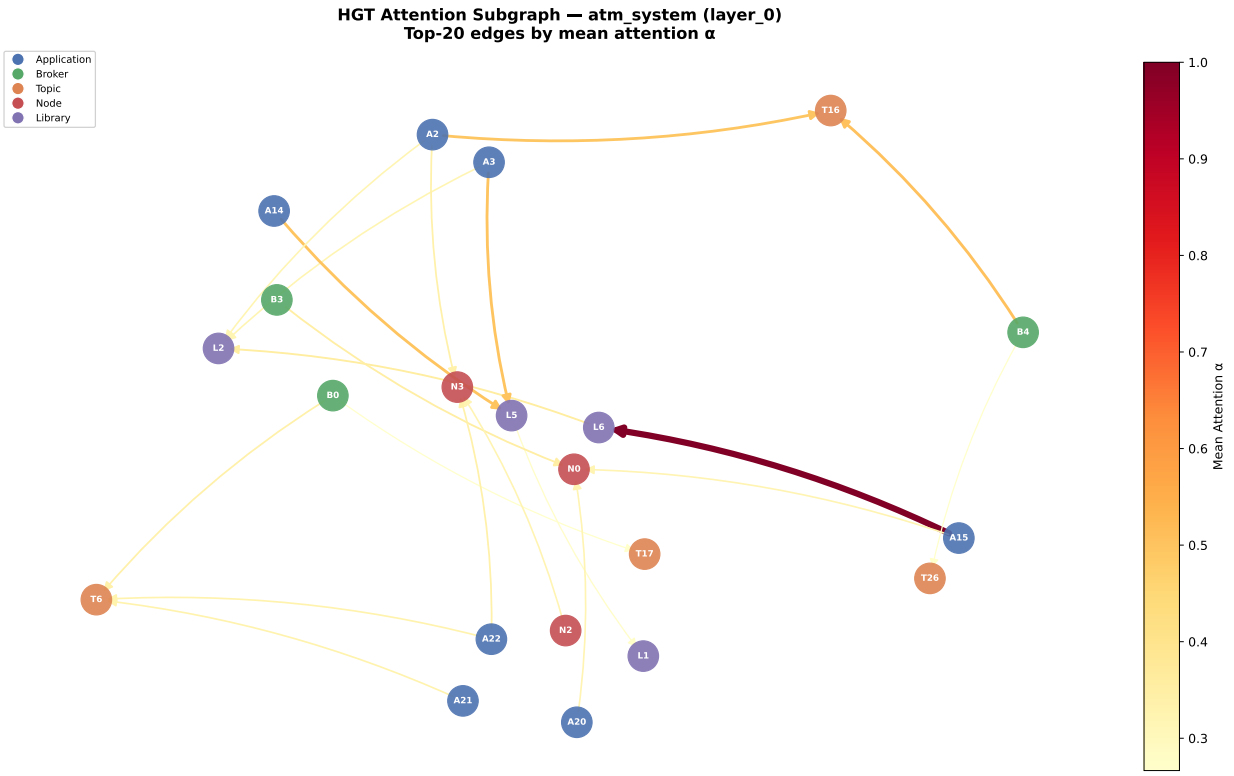

Figure 5: Relational attention extracted from the trained HGT on the ATM case study. The highest-weighted channels are `USES` (Application → shared Library) and `ROUTES` (Broker → Topic) rather than direct publish/subscribe edges, matching the shared-library blast-radius and broker-centrality pathways found by the edge-removal analysis.

Table 14: Per-stage cost of the deployed (inference) path on generated systems of increasing size, CPU, median of three runs with a warm-up pass excluded. Training is not included: it is a one-off cost paid once over the corpus, not per analysed system.

$\|V\|$	$\|E\|$	**Analyse (s)**	**Graph→tensor (s)**	**HGT forward (ms)**	**Analyse : forward**
249	1,127	0.27	0.011	22.5	12×
499	2,402	0.95	0.022	15.7	61×
999	6,422	4.74	0.055	36.7	129×
1,998	19,301	23.83	0.153	43.7	**545**×

1,998. Any effort spent making this framework scale should therefore go to the classical graph metrics, not to the model: betweenness is $O(|V||E|)$ and the directed articulation score and closeness are $O(|V|(|V|+|E|))$, giving an overall $O(|V|^2+|V||E|)$. One mitigation already ships, with the connectivity-degradation term switching to deterministic top-50 core sampling above $|V| = 300$.

Cost tracks edges, not components. The corpus scenarios make this concrete: the 520-component Enterprise mesh takes 27.2 s to analyse while a *denser-in-name-only* 999-component generated system takes 4.7 s, because the former carries 3,245 structural edges against a far sparser fan-out per component. Practitioners sizing a CI/CD budget should estimate from $|E|$, or from $|V| \cdot |E|$, rather than from component count. Within the paper's corpus the whole gate — analysis plus all 18 evaluated detectors — runs in 0.02 s (29 components) to 27.4 s (520 components), comfortably inside a pull-request budget.

What we have not measured, and do not claim. Nothing above 2,000 components has been timed, and nothing here should be extrapolated past it; all figures are single-threaded CPU, with no GPU measurement; and one detector, `DEEP_PIPELINE`, is excluded throughout because exhaustive source-to-sink path enumeration does not terminate within ten minutes even at 50 applications (Section 7.3). That detector is the one component of the framework known *not* to scale, and its exclusion is a limitation rather than a tuning choice. Training cost, for completeness: the full seven-variant, eight-fold, five-seed leave-one-scenario-out sweep reported in Table 9 took approximately 45 minutes per learned variant on CPU.

7.5. RQ4: Real-World Distributed Software Architecture Validation

To assess external validity on authentic production architectures, we evaluated SaG on three open-source distributed systems.

Table 15: Empirical validation on authentic real-world distributed software architectures.

Real-World Architecture	$\|V\|$	$\|V_{\mathbf{app}}\|$	Spearman ρ	Kendall τ	$F_1@K$	Tie-Robust F_1	Non-Zero	Gain
Autoware.universe (ROS 2)	75	32	**0.688 ± 0.009**	0.517	**0.800**	0.800	19 / 32	+0.36
Cloud Microservices Mesh	60	22	**0.778 ± 0.001**	0.639	**1.000**	0.760	8 / 22	+0.01
Train-Ticket Booking Mesh	90	41	**0.759 ± 0.001**	0.605	**1.000**	0.810	14 / 41	+0.26

Key Insights for RQ4:

1. **Strong Real-World Rank Agreement:** SaG achieves high rank correlation on Cloud Microservices ($\rho = 0.778$) and Train-Ticket ($\rho = 0.759$), and solid agreement on Autoware.universe ($\rho = 0.688$).
2. **Critical-Set Containment:** Every single application with non-zero cascading impact in Cloud Microservices and Train-Ticket is successfully captured within the predicted top-K critical set (tie-robust $F_1@K = 0.760$ and 0.810).
3. **Substantial Predictive Gain:** SaG outperforms raw degree centrality by +0.360 on Autoware and +0.264 on Train-Ticket, demonstrating that typed dependency derivation captures critical architectural semantics beyond superficial connectivity.

8. Discussion, Threats to Validity, and Conclusion

8.1. Discussion and Practical Implications

Our empirical findings provide clear guidance for software architects and reliability engineers:

- **When to Use Graph Learning — and When Not To:** Heterogeneous GNNs provide the greatest return when evaluating new, unseen system architectures out of distribution ($\rho = 0.608$, HGL-QoS) and when identifying the critical top-K shortlist for architectural hardening ($F_1@K = 0.414$). But the honest comparison is against a training-free QoS-weighted centrality baseline, and there the margin is +0.037 and not statistically significant ($p = 0.74$, Table 9). A team that needs a critical-component ranking and nothing else should use Topo-QoS: it costs no training, no corpus, and no checkpoint. The learned model earns its cost when the typed attention, the multi-task reliability/maintainability

outputs, or the relationship-level criticality head are wanted alongside the ranking — and when the deployment target differs structurally from the training corpus, where the untyped alternative collapses ($\rho = 0.086$) and the typed QoS-aware model does not. Between the two typed variants: HGL for in-distribution accuracy, HGL-QoS otherwise (Section 7.3).

- **When to Use Interpretable Attribution:** For root-cause diagnosis and remediation planning, the deterministic RM profile is indispensable. Distinguishing whether a service is critical due to single-point-of-failure exposure (Availability) or wide error propagation (Fault Tolerance) directly dictates whether to introduce load-balanced replicas or circuit-breaker policies.

8.2. Threats to Validity

- **Construct Validity:** Our ground truth is generated via discrete-event cascade simulation on structural models rather than observing live production outages. To characterise construct divergence we evaluated three independent simulation engines (`FaultInjector`, `FailureSimulator`, `MessageFlowSimulator`). On *ordering* the evidence is genuinely convergent: the behavioural queue-flow simulator and the topological cascade oracle agree at $\rho = 0.883$, never below 0.756 on any scenario, despite being built on different principles. On *critical-set membership* it is not: top-K Jaccard is 0.31–0.42 across pairs against 0.111 expected by chance, and this is neither a tie-breaking artifact (≤ 0.005 change under a tie-robust cut) nor purely construct divergence — the labeler compared against its own reruns reaches only 0.44 in its worst scenario, so the statistic is unstable at this K for any oracle. We also could not close the gap by weighting: applying the same $w(t)$ to both topological oracles changes their agreement by $\Delta\rho = -0.009$, a null result we report rather than omit. A further 30–47% of components per scenario carry no simulated ground truth at all, because the cascade model cannot express direct Topic or Node failure. Accordingly, results established against one oracle should not be read as claims about another, and none of them are claims about observed production outages: the reported correlations measure surrogate fidelity to a simulator, not predictive accuracy against deployed systems.

- **Internal Validity:** Potential feature leakage is prevented by strict graph view separation: predictors operate on G_{analysis}, whereas ground-truth simulators operate on $G_{\text{structural}}$. We additionally identified and fixed a normalization defect in the code-quality scoring pipeline: a min-max helper previously assigned the maximal Code Quality Penalty to any zero-variance population, which is correct for one genuine node but was indistinguishable from "many nodes carrying no code-quality data at all" — the exact shape of every Library in our three real-world scenarios, which carry no source-level `code_metrics`. The fix (scoring an undifferentiated multi-node population as zero rather than maximal penalty) changes Library Maintainability tier classifications in the real-world corpus but, as expected for a fix confined to a population with no variance to rank, leaves Application-population rank correlation against $I^*(v)$ effectively unchanged (Table 15: $\Delta\rho \in \{-0.003, 0.000, 0.000\}$ across the three real-world scenarios).

- **External Validity:** While our corpus spans ten distinct system domains and three authentic open-source architectures (ROS 2 and microservices), future work should evaluate larger enterprise deployments with hundreds of microservices.

- **Conclusion Validity:** Given the heavy-tailed, non-normal distribution of cascading failure impacts, all statistical comparisons utilize non-parametric rank correlation (Spearman ρ, Kendall τ), bootstrap confidence intervals ($B = 2,000$), and paired Wilcoxon signed-rank tests. Two hazards deserve explicit statement because both changed conclusions in this study rather than merely threatening to.

  First, *aggregation across node types.* The RM composite's rank correlation pooled across types ($\rho = 0.028$) falls outside the range spanned by its own per-type correlations ($\rho \in [0.14, 0.50]$) — a Simpson's-paradox effect. Reported pooled, the same LOSO experiment placed the RM baseline at $\rho = -0.014$ and the untyped GL model at 0.381; reported on the Application population it places them at $+0.190$ and 0.086 respectively, reversing their order. Every figure in this paper is therefore scored on a single stated population (Section 6.3), and we quote no pooled cross-type correlation.

Second, *substrate*. An earlier version of the RM ablations in Section 7.3 scored $Q(v)$ from features restricted to the Application–Library `DEPENDS_ON` projection — the substrate built for GNN feature/label alignment — on which no Application is an articulation point and no incident edge is a bridge in six of eight scenarios. Four of Availability's five terms vanish there, leaving $A(v)$ (roughly 0.51 of the composite) constant, and all three affected sweeps consequently reported negative ρ for a baseline that is in fact weakly positive. The ablations are now computed through the full analysis pipeline. We record this because the failure mode is not visible in the output: a degenerate score produces plausible-looking correlations, and only checking the variance of each term exposed it.

8.3. Limitations and Future Work

1. **Safety and Security Integration:** SaG currently focuses on Reliability and Maintainability. Incorporating formal hazard categories (e.g., ISO 26262 ASIL ratings) and security threat models into the multigraph schema represents an important direction for future research.
2. **Hardware-in-the-Loop (HIL) Validation:** Validating SaG's predictions against real-time physical fault-injection testbeds in cyber-physical environments.
3. **Automated Architectural Refactoring:** Extending SaG from predictive analysis to prescriptive synthesis—automatically generating pull requests that reconfigure QoS policies and add redundancy.

8.4. Conclusion

We presented **Software-as-a-Graph (SaG)**, a pre-deployment Static System Analysis framework that models complex distributed pub-sub and microservice architectures as typed, weighted multigraphs. By combining relation-specific Heterogeneous Graph Transformers with an interpretable Reliability–Maintainability quality attribution model, SaG bridges the Architecture–Code Gap, forecasting cascading failure risks before systems are deployed.

Our empirical results across synthetic and authentic open-source distributed systems show that relation-specific typing is what makes graph learning transfer to unseen architectures: the typed model reaches $\rho = 0.608$ where its untyped counterpart collapses to 0.086, an advantage significant across all eight folds. We are equally explicit about the boundary of that result — against a training-free QoS-weighted structural baseline the typed model's ranking advantage is $+0.037$ and not statistically significant, so the case for graph learning here rests on what it supplies beyond a ranking (typed attention, multi-task quality outputs, relationship-level criticality) rather than on ranking accuracy alone.

CRediT authorship contribution statement

[Omitted for double-anonymised review. To be completed on acceptance with the standard CRediT roles: Conceptualization; Methodology; Software; Validation; Formal analysis; Investigation; Data curation; Writing – original draft; Writing – review and editing; Visualization; Supervision.]

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

[Omitted for double-anonymised review.]

Data availability

The replication package—including the seven synthetic scenario datasets and the configurations that generate them, the topology generator, the three simulation harnesses, the real-world architecture adapters, and every analysis script behind the reported tables and figures—will be made openly available upon publication. The synthetic corpus is regenerable rather than merely archived: each dataset carries its seed and a SHA-256 digest in a committed manifest, and a regression test asserts byte-identical regeneration from the configurations (Section 6.1.1). Every table and figure in this paper is produced from a committed artifact by a script in that package; none of the reported values is transcribed by hand.

Declaration of generative AI use

[To be completed by the authors in accordance with the journal's policy.]

References

[1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot operating system 2: Design, architecture, and uses in the wild, Science Robotics 7 (66) (2022) eabm6074.

[2] J. Kreps, N. Narkhede, J. Rao, Kafka: A distributed messaging system for log processing, in: Proc. 6th Int. Workshop on Networking Meets Databases (NetDB), 2011.

[3] Object Management Group, Data distribution service (dds), Tech. Rep. formal/2015-04-10, version 1.4, Object Management Group (2015).

[4] OASIS, MQTT version 5.0, OASIS Standard (2019).

[5] P. T. Eugster, P. A. Felber, R. Guerraoui, A.-M. Kermarrec, The many faces of publish/subscribe, ACM Computing Surveys 35 (2) (2003) 114–131.

[6] International Organization for Standardization, ISO/IEC 25010:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — product quality model, Tech. rep., International Organization for Standardization (2023).

[7] International Organization for Standardization, ISO/IEC 25019:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality-in-use model, Tech. rep., International Organization for Standardization (2023).

[8] T. J. McCabe, A complexity measure, IEEE Transactions on Software Engineering SE-2 (4) (1976) 308–320.

[9] S. R. Chidamber, C. F. Kemerer, A metrics suite for object oriented design, IEEE Transactions on Software Engineering 20 (6) (1994) 476–493.

[10] N. Fenton, J. Bieman, Software Metrics: A Rigorous and Practical Approach, 3rd Edition, CRC Press, 2014.

[11] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, C. Rosenthal, Chaos engineering, IEEE Software 33 (3) (2016) 35–41.

[12] L. C. Freeman, A set of measures of centrality based on betweenness, Sociometry 40 (1) (1977) 35–41.

[13] S. Brin, L. Page, The anatomy of a large-scale hypertextual web search engine, Computer Networks and ISDN Systems 30 (1–7) (1998) 107–117.

[14] U. Brandes, A faster algorithm for betweenness centrality, Journal of Mathematical Sociology 25 (2) (2001) 163–177.

[15] M. E. J. Newman, Networks: An Introduction, Oxford University Press, 2010.

[16] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: Proc. Int. Conf. on Learning Representations (ICLR), 2017.

[17] W. L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: Advances in Neural Information Processing Systems 30 (NeurIPS), 2017, pp. 1024–1034.

[18] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: Proc. Int. Conf. on Learning Representations (ICLR), 2018.

[19] V. R. Basili, L. C. Briand, W. L. Melo, A validation of object-oriented design metrics as quality indicators, IEEE Transactions on Software Engineering 22 (10) (1996) 751–761.

[20] N. Nagappan, T. Ball, Static analysis tools as early indicators of pre-release defect density, in: Proc. 27th Int. Conf. on Software Engineering (ICSE), 2005, pp. 580–586.

[21] T. Zimmermann, R. Premraj, A. Zeller, Predicting defects for Eclipse, in: Proc. 3rd Int. Workshop on Predictor Models in Software Engineering (PROMISE), 2007.

[22] T. Menzies, J. Greenwald, A. Frank, Data mining static code attributes to learn defect predictors, IEEE Transactions on Software Engineering 33 (1) (2007) 2–13.

[23] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Toward a catalogue of architectural bad smells, in: Proc. 5th Int. Conf. on the Quality of Software Architectures (QoSA), LNCS 5581, 2009, pp. 146–162.

[24] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Identifying architectural bad smells, in: Proc. 13th European Conf. on Software Maintenance and Reengineering (CSMR), 2009, pp. 255–258.

[25] D. Taibi, V. Lenarduzzi, On the definition of microservice bad smells, IEEE Software 35 (3) (2018) 56–62.

[26] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, L. Safina, Microservices: Yesterday, today, and tomorrow, in: Present and Ulterior Software Engineering, Springer, 2017, pp. 195–216.

[27] W. Cunningham, The WyCash portfolio management system, in: Addendum to the Proc. Conf. on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA), 1992, pp. 29–30.

[28] Z. Li, P. Avgeriou, P. Liang, A systematic mapping study on technical debt and its management, Journal of Systems and Software 101 (2015) 193–220.

[29] J. Humble, D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation, Addison-Wesley, 2010.

[30] L. Chen, Continuous delivery: Huge benefits, but challenges too, IEEE Software 32 (2) (2015) 50–54.

[31] R. Albert, H. Jeong, A.-L. Barabási, Error and attack tolerance of complex networks, Nature 406 (2000) 378–382.

[32] A. E. Motter, Y.-C. Lai, Cascade-based attacks on complex networks, Physical Review E 66 (2002) 065102(R).

[33] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, S. Havlin, Catastrophic cascade of failures in interdependent networks, Nature 464 (2010) 1025–1028.

[34] C. Fan, L. Zeng, Y. Sun, Y.-Y. Liu, Finding key players in complex networks through deep reinforcement learning, Nature Machine Intelligence 2 (2020) 317–324.

[35] C. Fan, L. Zeng, Y. Ding, M. Chen, Y. Sun, Z. Liu, Learning to identify high betweenness centrality nodes from scratch: A novel graph neural network approach, in: Proc. 28th ACM Int. Conf. on Information and Knowledge Management (CIKM), 2019, pp. 559–568.

[36] A. Varbella, K. Amara, M. El-Assady, B. Gjorgiev, G. Sansavini, PowerGraph: A power grid benchmark dataset for graph neural networks, in: Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track, 2024, arXiv:2402.02827.

[37] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, M. Welling, Modeling relational data with graph convolutional networks, in: Proc. European Semantic Web Conference (ESWC), 2018, pp. 593–607.

[38] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, P. S. Yu, Heterogeneous graph attention network, in: Proc. The Web Conference (WWW), 2019, pp. 2022–2032.

[39] Z. Hu, Y. Dong, K. Wang, Y. Sun, Heterogeneous graph transformer, in: Proc. The Web Conference (WWW), 2020, pp. 2704–2710.

[40] X. Fu, J. Zhang, Z. Meng, I. King, MAGNN: Metapath aggregated graph neural network for heterogeneous graph embedding, in: Proc. The Web Conference (WWW), 2020, pp. 2331–2341.

[41] International Organization for Standardization, ISO/IEC 25023:2016 — systems and software engineering — systems and software quality requirements and evaluation (square) — measurement of system and software product quality, Tech. rep., International Organization for Standardization (2016).

[42] International Organization for Standardization, ISO/IEC 25021:2012 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality measure elements, Tech. rep., International Organization for Standardization (2012).

[43] T. L. Saaty, The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation, McGraw-Hill, 1980.

[44] Team SimPy, Simpy: Event discrete simulation for Python, `https://simpy.readthedocs.io` (2020).

[45] F. Xia, T.-Y. Liu, J. Wang, W.-S. Zhang, H. Li, Listwise approach to learning to rank: Theory and algorithm, in: Proc. 25th Int. Conf. on Machine Learning (ICML), 2008, pp. 1192–1199.

[46] S. Kato, S. Tokunaga, Y. Maruyama, S. Maeda, M. Hirabayashi, Y. Kitsukawa, A. Monrroy, T. Ando, Y. Fujii, T. Azumi, Autoware on board: Enabling autonomous vehicles with embedded systems, in: Proc. ACM/IEEE 9th Int. Conf. on Cyber-Physical Systems (ICCPS), 2018, pp. 287–296.

[47] Google Cloud Platform, Online boutique: A cloud-native microservices demo application, Software artifact (2024).

[48] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, D. Ding, Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study, IEEE Transactions on Software Engineering 47 (2) (2021) 243–260.

[49] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83.

[50] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, 1993.

[51] C. Spearman, The proof and measurement of association between two things, American Journal of Psychology 15 (1) (1904) 72–101.